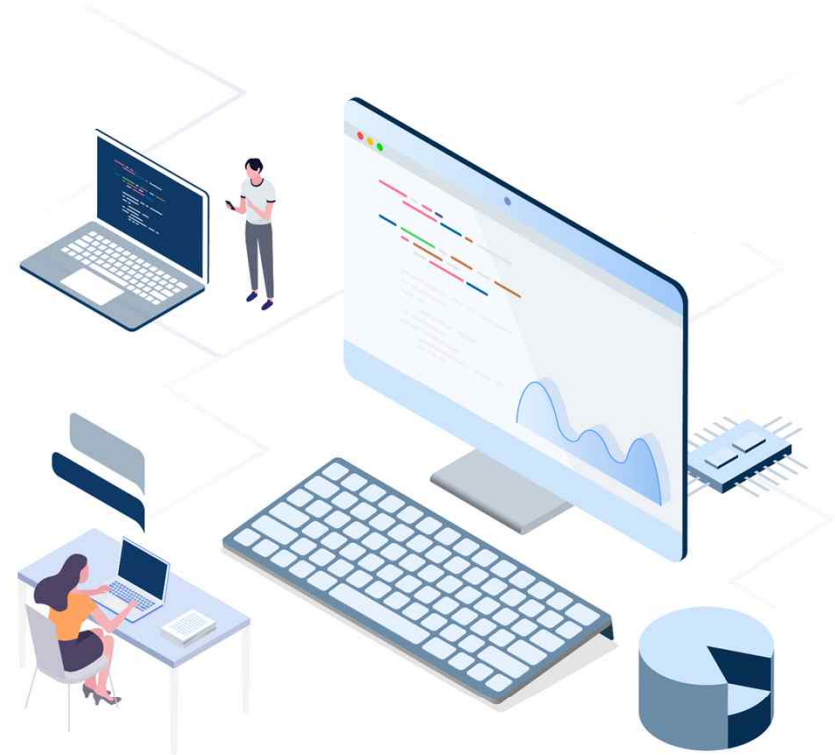


웹 프로그래밍

[Web Programming]

Ch 13. Spring Framework, SpringBoot, WebFlux, WebClient, CRUD, Multimedia Streaming

13.1	Spring Framework – MVC (Model, View, Controller)
13.2	Kotlin SpringBoot 기반 CRUD 서비스 구현 사용자 (user) 계정관리 웹 서비스 구현
13.3	Spring WebFlux, WebClient
13.4	Spring WebFlux와 WebClient 기반 Multimedia (audio/video) Streaming Service



컴퓨터학부 김영탁 교수

학습내용

- ◆ Kotlin/Java Spring Framework
- ◆ Spring MVC (Model, View, Controller) 구조
- ◆ Kotlin Spring Boot 기반 CRUD (Create, Read, Update, Delete) 웹 서비스 구현
- ◆ Kotlin Spring 기반 사용자 (User) 관리 웹 응용 서비스 구현
- ◆ Spring WebFlux, WebClient
- ◆ Spring WebFlux와 WebClient 기반 Multimedia Streaming Service

Spring Framework, Spring Boot, Kotlin Spring

Spring Framework

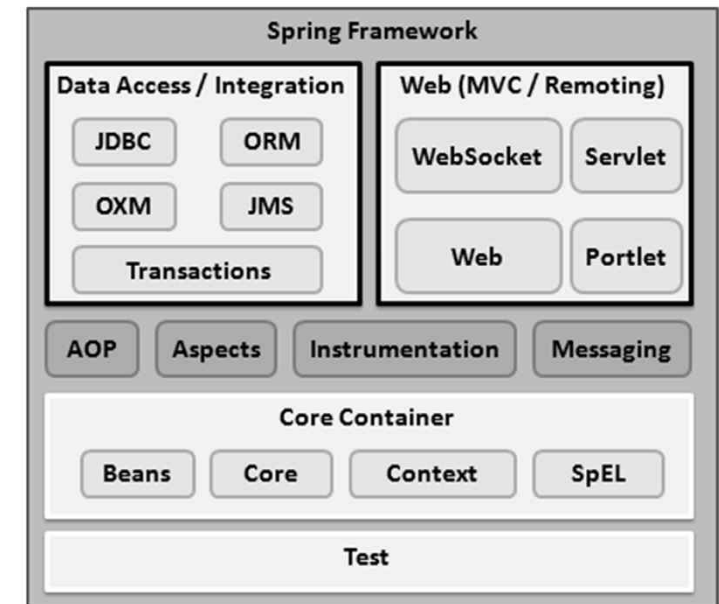
◆ Spring Framework

- Spring framework tutorial,
https://www.youtube.com/watch?v=MHoFSfXGX_A&list=PLw_k9CF7hBpJJsRWAhwSrDIWAzuMV0irI
- 엔터프라이즈용 Java 애플리케이션 개발을 편하게 할 수 있게 해주는 오픈소스 경량급 애플리케이션 프레임워크
- 엔터프라이즈 비즈니스 어플리케이션을 개발하기 위하여 필요한 기능을 다양하게 제공하기 때문에 개발 초기에 기본적인 설정과 적용시킬 기술들을 잘 선택할 수 있으며, 기술보다는 애플리케이션의 로직 자체에 더 집중하여 비즈니스 로직을 구현할 수 있음
- Spring은 오픈소스로서 어떤 개인 및 기업도 스프링을 사용하여 웹 애플리케이션을 개발을 할 수 있으며, 필요하다면 스프링의 코드를 일부 수정하여 사용할 수 있음
- Spring은 오픈소스 프레임워크이지만, 안정적인 개발과 개선이 보장됨
- 기존 Spring 대신 사용하던 기술들과 비교하여, Spring을 사용했을 때에 개발자가 작성해야 할 코드가 상대적으로 단순하며, 따라서 경량급이라 함
- Spring을 사용함으로써 기존 기술 기반의 엔터프라이즈 애플리케이션 개발에서 불가피하게 작성해야만 했던 불필요하게 복잡한 코드를 제거하여 코드의 복잡성을 낮출 수 있음
- 엔터프라이즈 애플리케이션을 개발하는 데에 있어 필요한 모든 업무 분야 및 모든 기술과 관련된 코드들의 뼈대를 제공하는 애플리케이션 프레임워크 임

Spring Framework 특징

◆ Spring Framework 특징

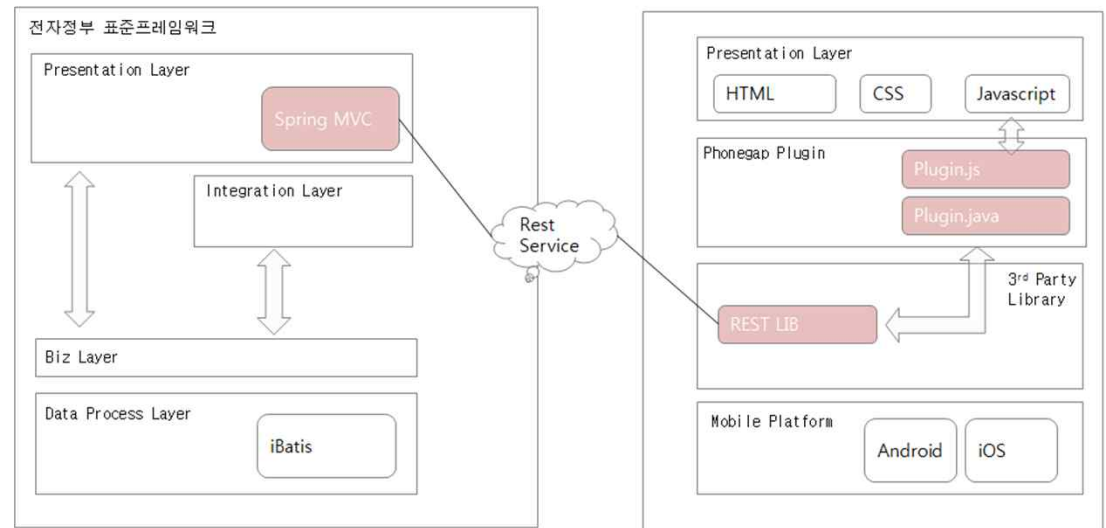
- 스프링의 가장 큰 특징은 Plain Old Java Object (POJO)를 기반으로 하며, plain old Java object만을 사용한다는 것은 Java 및 Java의 스펙에 정의된 기술만 사용한다는 의미임
- 즉, 어떤 객체가 외부의 라이브러리나 외부의 모듈을 가져와서 사용하고 있지 않고, 다른 기술을 사용하지 않는 순수한 Java만을 사용하여 만든 객체를 사용하며, 따라서 특정 기술이나 환경에 종속되지 않음
- POJO (Plain Old Java Object)프로그래밍을 위해 Spring은 IoC (inversion of control)/DI(dependency injection), AOP(Aspect Oriented Programming), PSA(Portable Service Abstraction)를 지원함
- 스프링은 데이터베이스 서비스를 추상화한 인터페이스인 JDBC(Java DataBase Connectivity) 를 제공하며, JDBC를 기반으로 하여 데이터베이스 접근 코드를 작성해두면, 이후에 데이터베이스를 바꾸어도 기존에 작성한 데이터베이스 접근 로직을 그대로 사용할 수 있음
- 이러한 JDBC처럼 특정 기술과 관련된 서비스를 추상화하여 일관된 방식으로 사용될 수 있도록 한 것을 PSA(Portable Service Abstraction, 일관된 서비스 추상화)라고 함



전자정부 표준 프레임워크

◆ 전자정부 표준 프레임워크

- Server 측의 presentation layer에 Spring MVC (Model-View-Controller)
- 전자정부 표준 프레임워크 기반의 서버 애플리케이션과 모바일 클라이언트는 웹서비스를 통해 Loosely Coupling 됨
- 웹서비스는 REST Service 프로토콜을 이용하여 클라이언트와 통신
- 웹 서비스를 이용하는 모바일 클라이언트는 전자정부 표준 프레임워크 REST 서비스 뿐 아니라 표준 REST 서비스를 구현한 서버와의 통신이 가능함.



사용된 오픈소스

서비스	버전	라이선스
Android SDK	revision 18	Apache License 2.0
Android Platform	Android 2.2 Froyo 이상	
PhoneGap(cordova)	Cordova 4.3.0	MIT License
JavaScript Framework	전자정부 표준프레임워크 사용자 경험(UX) 기능	Apache License 2.0
iOS SDK	Xcode 6.3.0	
iOS Platform	iOS 6.0이상	
ASIHHttpRequest	1.8.1	Apache License 2.0

Spring MVC 처리 순서



Spring Framework 기반 Web의 MVC (Model-View-Controller)

◆ Spring MVC 구조 및 동작 흐름

- Spring Framework의 Web 서비스에서 MVC (Model-View-Controller) 아키텍처는 웹 애플리케이션 개발에서 중요한 역할을 함.
- 이 아키텍처는 애플리케이션의 구조를 모델, 뷰, 컨트롤러 세 가지 주요 컴포넌트로 분리하여 유지 보수와 확장성을 높이는 구조임.

◆ Spring MVC의 동작 흐름

- 클라이언트 요청: 사용자가 웹 페이지에서 특정 URL을 요청하면, Spring의 DispatcherServlet이 해당 요청을 수신.
- 요청 처리: DispatcherServlet은 요청 URL을 분석하여 적합한 컨트롤러의 메서드를 찾고 요청을 전달.
- 비즈니스 로직 수행: 컨트롤러는 비즈니스 로직을 수행할 필요가 있다면, 서비스 계층을 통해 모델과 상호작용 실행.
- 뷰 반환: 컨트롤러가 데이터를 준비하면, 이를 뷰로 전달.
- 뷰 렌더링 및 응답 반환: 뷰 템플릿 엔진이 데이터를 바탕으로 최종 HTML을 생성하고, 클라이언트에게 응답을 반환.

◆ Spring MVC의 장점

- 관심사 분리: MVC 구조는 애플리케이션을 역할별로 분리하여 코드의 재사용성과 유지보수성을 높임.
- 테스트 용이성: 컨트롤러와 서비스, 모델이 분리되어 있어 각 부분의 테스트가 독립적으로 가능해짐.
- 확장성: 비즈니스 로직, UI, 데이터 관리가 분리되어 있어 요구사항 변경에 유연하게 대처할 수 있음.

Spring Framework MVC (Model, View, Controller)

◆ Spring MVC Model (Service, Repository)

- MVC의 Model은 애플리케이션의 데이터를 관리하는 역할을 하며, 데이터의 상태와 비즈니스 로직을 처리함. 예를 들어, 데이터베이스와의 상호작용을 통해 정보를 가져오거나 저장하는 작업이 여기서 이루어짐.
- 모델은 도메인 객체로 구성되며, 예를 들어 User, Order와 같은 데이터 구조체가 모델에 포함될 수 있음. 또한, 데이터베이스와 상호작용하는 Repository나 Service 계층이 모델에 해당함.

```
import org.springframework.stereotype.Service
import org.springframework.data.repository.findByIdOrNull
import org.springframework.stereotype.Repository
import org.springframework.data.jpa.repository.JpaRepository
```

@Service

```
class UserService(private val userRepository: UserRepository) {
    fun findUserById(id: Long): User? {
        return userRepository.findByIdOrNull(id)
    }
}
```

@Repository

```
interface UserRepository : JpaRepository<User, Long>
```

```
import javax.persistence.Entity
import javax.persistence.GeneratedValue
import javax.persistence.GenerationType
import javax.persistence.Id
```

@Entity

```
data class User(
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    val id: Long = 0,
    val name: String,
    val email: String
)
```

◆ Spring MVC View

- MVC의 View는 사용자에게 데이터를 표시하는 역할을 담당함. 뷰는 컨트롤러에서 넘겨준 데이터를 사용하여 사용자에게 정보를 표시함.
- Spring MVC에서는 JSP(JavaServer Pages), Thymeleaf, Freemarker 등의 템플릿 엔진을 사용하여 뷰를 생성할 수 있음.
최근에는 SPA (Single Page Application)를 위해 REST API로 데이터를 제공하고, 프론트 엔드 프레임워크(React, Vue.js 등)로 뷰를 관리하기도 함.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>User View</title>
</head>
<body>
  <h1>User Information</h1>
  <p>Name: <span th:text="${user.name}"></span></p>
  <p>Email: <span th:text="${user.email}"></span></p>
</body>
</html>
```

◆ Spring MVC Controller

- MVC의 Controller는 클라이언트의 요청을 받아 모델을 통해 데이터를 처리하고, 결과를 뷰에 전달하는 역할을 함. 즉, 클라이언트의 요청(Request)과 응답(Response)을 관리하고 애플리케이션의 흐름을 제어함.
- 컨트롤러에서는 주로 Spring의 @Controller나 @RestController annotation을 사용하여 요청을 처리하는 메서드를 정의함.
이때 @RequestMapping, @GetMapping, @PostMapping 등을 통해 요청 URL과 매핑하며, 모델과 뷰의 연결 역할을 함.

```
import org.springframework.stereotype.Controller
import org.springframework.ui.Model
import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.PathVariable
import org.springframework.web.bind.annotation.RequestMapping
```

@Controller

@RequestMapping("/users")

```
class UserController(private val userService: UserService) {
```

@GetMapping("/{id}")

```
fun getUser(@PathVariable id: Long, model: Model): String {
    val user = userService.findUserById(id)
    model.addAttribute("user", user)
    return "userView" // Thymeleaf 템플릿 이름
}
}
```

Spring Framework에서 사용되는 Annotation

애너테이션	설명	사용 예시
@Controller	Spring MVC에서 클래스를 컨트롤러로 정의하며, 요청을 처리하는 역할을 함	@Controller class UserController { ... }
@RestController	@Controller와 @ResponseBody를 함께 사용하는 효과로, REST API 용도	@RestController class UserApiController { ... }
@RequestMapping	클래스나 메서드에 URL 요청 경로를 매핑하여, 특정 요청에 대해 실행될 메서드를 지정	@RequestMapping("/users")
@GetMapping	HTTP GET 요청을 특정 메서드에 매핑함, 주로 데이터를 조회하는 요청에 사용	@GetMapping("/users/{id}") fun getUser(@PathVariable id: Long): User { ... }
@PostMapping	HTTP POST 요청을 특정 메서드에 매핑함, 주로 데이터를 생성하는 요청에 사용	@PostMapping("/users") fun createUser(@RequestBody user: User): Response { ... }
@PutMapping	HTTP PUT 요청을 특정 메서드에 매핑함, 주로 데이터를 업데이트하는 요청에 사용	@PutMapping("/users/{id}") fun updateUser(@PathVariable id: Long, @RequestBody user: User): Response { ... }
@DeleteMapping	HTTP DELETE 요청을 특정 메서드에 매핑함, 주로 데이터를 삭제하는 요청에 사용	@DeleteMapping("/users/{id}") fun deleteUser(@PathVariable id: Long): Response { ... }
@RequestParam	요청 파라미터 값을 메서드 파라미터에 바인딩하여 사용	fun getUser(@RequestParam name: String): User { ... }
@PathVariable	URL 경로에 포함된 변수를 메서드 파라미터에 바인딩하여 사용	fun getUser(@PathVariable id: Long): User { ... }
@RequestBody	HTTP 요청의 본문(body)를 자바 객체에 매핑하여 사용	fun createUser(@RequestBody user: User): Response { ... }
@ResponseBody	반환 값을 HTTP 응답의 본문(body)에 매핑하여 JSON이나 XML 형태로 반환	@GetMapping("/users/{id}") @ResponseBody fun getUser(@PathVariable id: Long): User { ... }
@ModelAttribute	요청 파라미터나 세션 데이터를 모델에 바인딩하여 폼 데이터 전송 시 주로 사용	fun updateUser(@ModelAttribute user: User): String { ... }
@RequestHeader	HTTP 요청 헤더 정보를 메서드 파라미터에 바인딩하여 사용	fun getUser(@RequestHeader("User-Agent") userAgent: String): User { ... }
@SessionAttributes	모델에 저장된 객체를 세션에 저장하여 여러 요청 간에 데이터를 공유	@SessionAttributes("user")
@CookieValue	쿠키 값을 메서드 파라미터에 바인딩하여 사용	fun getUser(@CookieValue("SESSIONID") sessionId: String): User { ... }
@ExceptionHandler	특정 예외 발생 시 호출되는 메서드를 정의	@ExceptionHandler(UserNotFoundException::class) fun handleNotFound(ex: Exception): Response { ... }
@CrossOrigin	CORS(Cross-Origin Resource Sharing) 설정을 위해 사용	@CrossOrigin(origins = ["http://example.com"])
@ResponseStatus	메서드가 반환하는 HTTP 상태 코드를 설정	@ResponseStatus(HttpStatus.CREATED) fun createUser(@RequestBody user: User): User { ... }

Spring Boot

◆ Spring Boot 개요

- 스프링 부트 (Spring Boot)는 스프링으로 애플리케이션을 만들 때에 필요한 설정을 간편하게 처리해주는 별도의 프레임워크임
- 스프링 부트를 사용하면 초기 설정을 간편하게 할 수 있는 것 외에도 몇 가지 장점을 제공함: 기존에는 배포를 할 때에 별도의 외장 웹 서버를 설치하고, 프로젝트를 War 파일로 빌드하여 배포를 진행했는데, 이러한 방식은 처리 속도가 느리며 번거롭다는 단점을 가짐.
- 반면, 스프링 부트는 **자체적인 웹 서버**를 내장하고 있어, 빠르고 간편하게 배포를 진행할 수 있음.
- 또한, Spring Boot를 사용하면 독립적으로 실행 가능한 Jar 파일로 프로젝트를 빌드할 수 있어, cloud service 및 docker와 같은 가상화 환경에 빠르게 배포할 수 있음.

◆ Spring Boot Tutorial, 참고자료

- inflearn.com/course/스프링-입문-스프링부트, 김영한 (우아한형제들 개발팀장), 스프링 입문 - 코드로 배우는 스프링부트, 웹MVC, DB 접근 기술, <https://www.inflearn.com/course/%EC%8A%A4%ED%94%84%EB%A7%81-%EC%9E%85%EB%AC%B8-%EC%8A%A4%ED%94%84%EB%A7%81%EB%B6%80%ED%8A%B8>
- Spring Boot: A Tutorial for Beginners | in28minutes | Ranga Karanam, <https://www.youtube.com/watch?v=zTAHY0SLQAg>
- 코딩의 신, Spring Boot으로 웹 출시까지 1. 환경 설정 -YouTube
- 프로젝트 생성하기 [스프링 + 코틀린 (Spring + Kotlin)] (youtube.com)
- Spring Boot with Kotlin & JUnit 5 -Tutorial 1 -Introduction (youtube.com)

Spring Boot 개요

◆ Spring Boot 개요

- <https://spring.io/projects/spring-boot>
- <https://spring.io/projects/spring-boot#learn>
- 스프링은 엔터프라이즈용 Java 애플리케이션 개발을 편하게 할 수 있게 해주는 오픈소스 경량급 애플리케이션 프레임워크 임
- 스프링은 POJO (Plain Old Java Object) 프로그래밍을 지향하는 특징을 가지며, 그를 위해 IoC/DI, AOP, PSA를 지원함
- IoC(inversion of control)는 객체 간의 의존 관계를 개발자가 아닌 스프링이 맺어주는 것을 의미함
- DI(dependency injection)는 의존 객체를 특정 클래스에서 직접 생성하는 것이 아니라, 외부로부터 주입해주는 것을 의미함
- AOP(Asspect Oriented Programming)는 애플리케이션 전반에 걸쳐 공통적으로 적용되는 코드들을 비즈니스 로직으로부터 분리해내는 것을 의미함
- PSA (Portable Service Abstraction)는 특정 기술과 관련된 서비스들을 추상화 하여 일관된 방식으로 사용할 수 있도록 한 것을 의미함
- Spring Boot는 Spring Application을 만들 때에 필요한 초기 설정을 간편하게 해주는 프레임워크 임
- Spring Initializer를 사용하면 스프링 프로젝트를 편리하게 생성할 수 있음

Spring Boot 기반의 웹 응용 서비스 구현

◆ Spring Boot Web App

- Spring Web Services Reference Documentation,
<https://docs.spring.io/spring-ws/docs/current/reference/html/>
- <https://www.youtube.com/watch?v=bxy2JgqgKDU>
- <https://vaadin.com/docs/latest/overview>
- [Spring Boot React Full Stack | Full Course -YouTube](#),
 Employee management app using React (open-source project created by Facebook)

◆ Building an Application with Spring Boot

- <https://spring.io/guides/gs/spring-boot/>
- <https://github.com/spring-guides/gs-spring-boot>
- simple web application with Spring Boot and add some useful services to it.
- <https://www.youtube.com/watch?v=YDRNMAJo0MA>

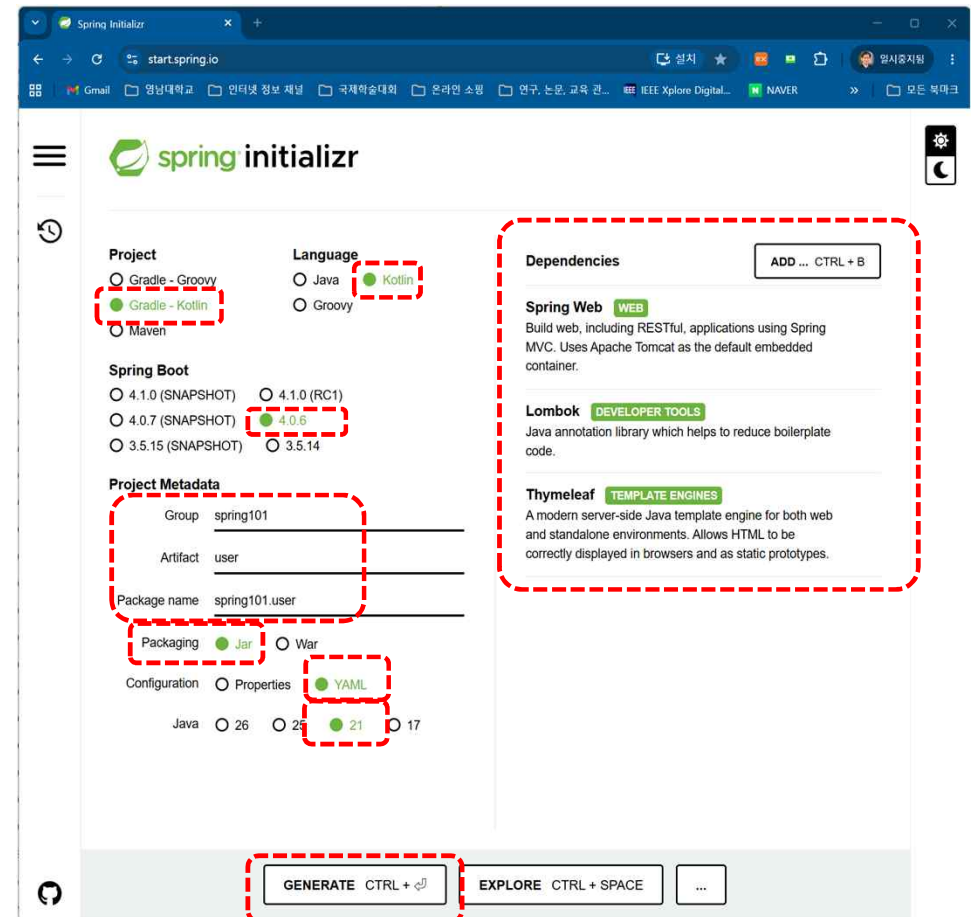
◆ Spring initializr

- <https://start.spring.io/>
- <https://while1.tistory.com/104>

Spring Boot 프로젝트 생성, Spring Initializer

◆ Spring Initializr

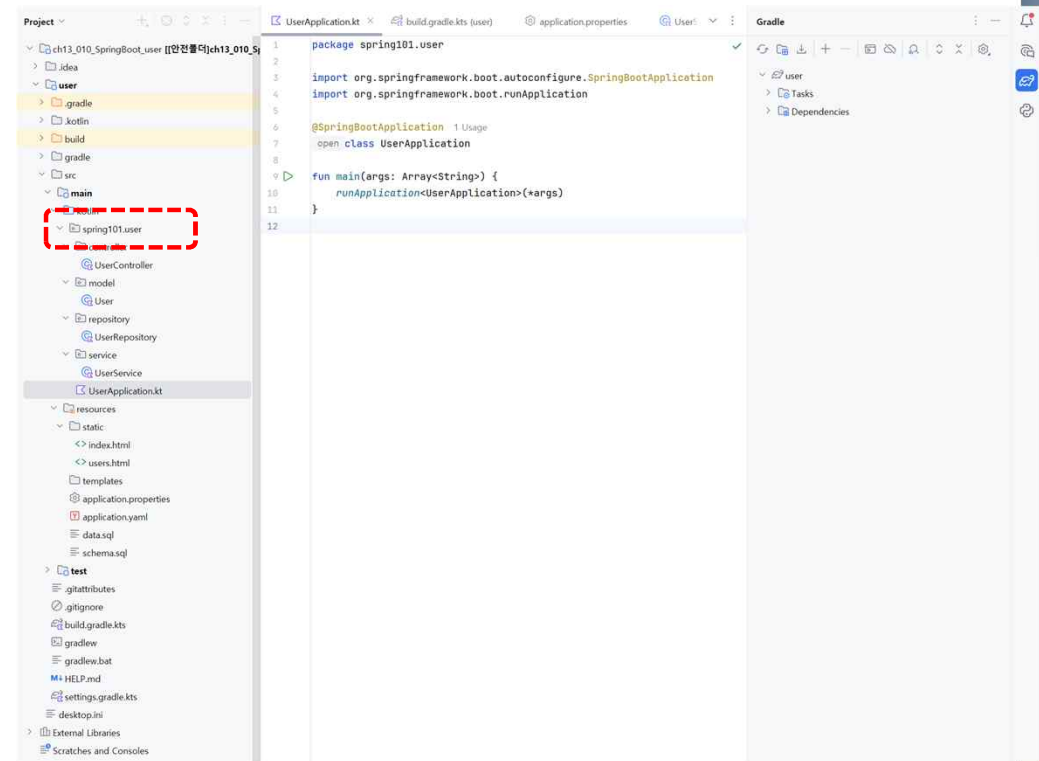
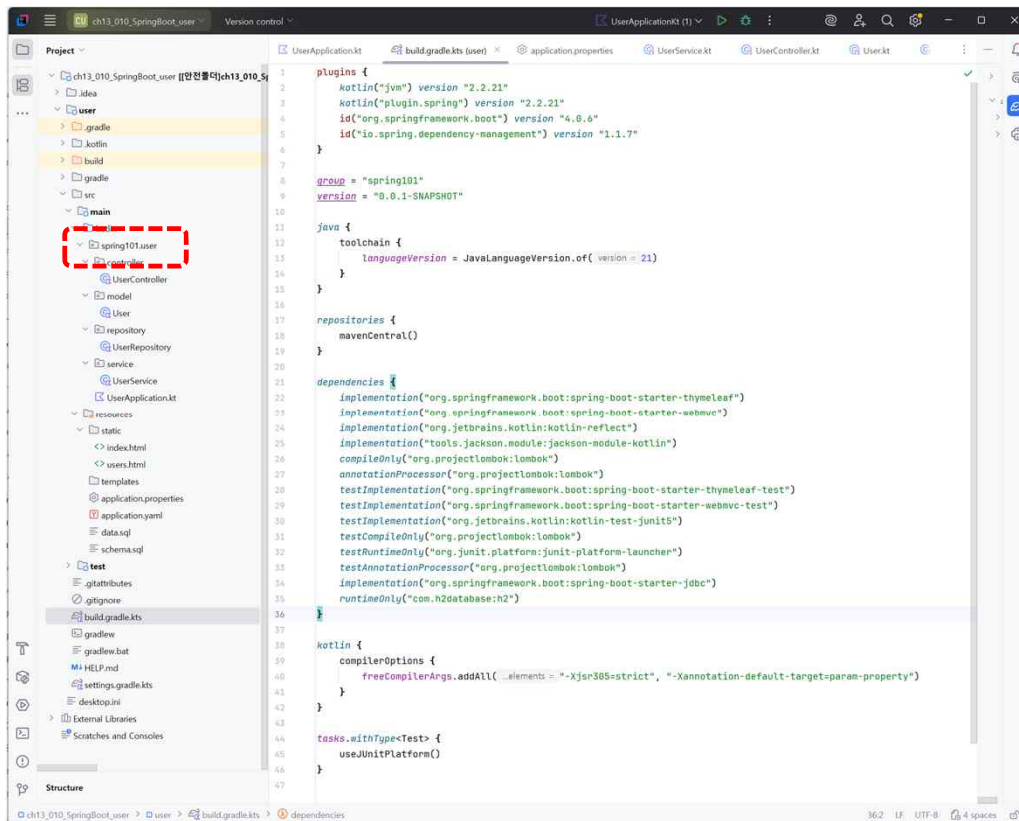
- <https://start.spring.io/>
- Project : Gradle-Kotlin
- Language : Kotlin
- Spring Boot 4.0.6
- Package name : **spring101.user**
- Java 21
- Dependencies :
 - Spring Web (Web)
 - Thymeleaf (Template Engines)



◆ IntelliJ Community에서 Spring Initializr 사용

- Community version에는 설치되어 있지 않으며, 따라서 위 site에서 생성하여야 함

D:\SpringBoot 디렉토리에 생성된 zip 파일 설치, 압축풀기 (이름을 Spring_101.user으로 설정) 및 IntelliJ에 load



SpringBoot 기반 MVC 구조의 User 관리 Web Service

◆ 전체 프로젝트 구조

- Model
- View
- Controller

◆ Model

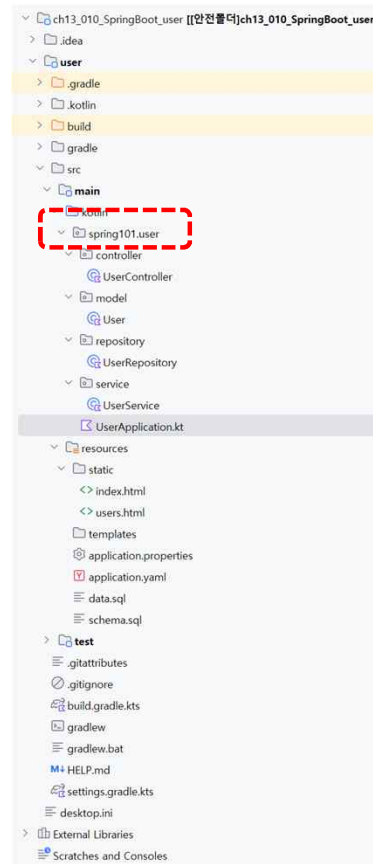
- User
- UserRepository
- UserService

◆ View

- users.html

◆ Controller

- UserController



User List

ID	Name	Email
1	Alice	alice@example.com
2	Bob	bob@example.com
3	Kim	kim@abc.com
4	Lee	lee@xyz.com
5	Park	park@yu.ac.kr
6	Choi	choi@sss.co.kr
7	Hong	hong@abc.ac.kr

build.gradle.kts

```

plugins {
    kotlin("jvm") version "2.2.21"
    kotlin("plugin.spring") version "2.2.21"
    id("org.springframework.boot") version "4.0.6"
    id("io.spring.dependency-management") version "1.1.7"
}

group = "spring101"
version = "0.0.1-SNAPSHOT"

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.springframework.boot:spring-boot-starter-thymeleaf")
    implementation("org.springframework.boot:spring-boot-starter-webmvc")
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    implementation("tools.jackson.module:jackson-module-kotlin")
    compileOnly("org.projectlombok:lombok")
    annotationProcessor("org.projectlombok:lombok")

```

```

    testImplementation("org.springframework.boot:spring-boot-starter-thymeleaf-test")
    testImplementation("org.springframework.boot:spring-boot-starter-webmvc-test")
    testImplementation("org.jetbrains.kotlin:kotlin-test-junit5")
    testCompileOnly("org.projectlombok:lombok")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    testAnnotationProcessor("org.projectlombok:lombok")
    implementation("org.springframework.boot:spring-boot-starter-jdbc")
    runtimeOnly("com.h2database:h2")
}

kotlin {
    compilerOptions {
        freeCompilerArgs.addAll("-Xjsr305=strict", "-Xannotation-default-target=param-property")
    }
}

tasks.withType<Test> {
    useJUnitPlatform()
}

```

User, UserService

```
/* spring101.user.model.User.kt */
package spring101.user.model

data class User(
    val id: Long = 0,
    val name: String = "",
    val email: String = ""
)
```

```
/* spring101.user.service.UserService.kt (1) */
package spring101.user.service

import spring101.user.model.User
import spring101.user.repository.UserRepository
import org.springframework.stereotype.Service
import org.slf4j.Logger
import org.slf4j.LoggerFactory

@Service
class UserService(private val userRepository: UserRepository) {
    private val logger: Logger = LoggerFactory.getLogger(UserService::class.java)
    fun findUser(id: Long): User? {
        logger.info("Finding user with id: $id")
        val user: User? = userRepository.findById(id)/* user 검색 로직 */
        if (user == null) {
            logger.warn("User with id $id not found")
        }
        return user
    }
}
```

```
/* spring101.user.service.UserService.kt (2) */

fun getAllUsers(): List<User> = userRepository.findAll()
fun getUserById(id: Long): User? = userRepository.findById(id)
fun createUser(user: User): User = userRepository.save(user)
fun updateUser(id: Long, updatedUser: User): User? {
    return if (userRepository.update(id, updatedUser) > 0) {
        updatedUser.copy(id = id)
    } else {
        null
    }
}

fun deleteUser(id: Long): Boolean {
    return userRepository.deleteByld(id) > 0
}
```

UserRepository

```
/* spring101.user.repository.UserRepository.kt (1) */

package spring101.user.repository

import spring101.user.model.User
import org.springframework.dao.EmptyResultDataAccessException
import org.springframework.jdbc.core.JdbcTemplate
import org.springframework.jdbc.core.RowMapper
import org.springframework.stereotype.Repository

@Repository
class UserRepository(private val jdbcTemplate: JdbcTemplate) {

    private val rowMapper = RowMapper<User> { rs, _ ->
        User(
            id = rs.getLong("id"),
            name = rs.getString("name"),
            email = rs.getString("email")
        )
    }

    fun findAll(): List<User> {
        return jdbcTemplate.query("SELECT * FROM users", rowMapper)
    }

    fun findById(id: Long): User? {
        return try {
            jdbcTemplate.queryForObject(
                "SELECT * FROM users WHERE id = ?",
                rowMapper,
                id
            )
        } catch (e: EmptyResultDataAccessException) {
            null
        }
    }
}
```

```
/* spring101.user.repository.UserRepository.kt (2) */

fun save(user: User): User {
    jdbcTemplate.update(
        "INSERT INTO users (name, email) VALUES (?, ?)",
        user.name,
        user.email
    )

    val id = jdbcTemplate.queryForObject(
        "SELECT IDENTITY()",
        Long::class.java
    ) ?: 0L

    return user.copy(id = id)
}

fun update(id: Long, user: User): Int {
    return jdbcTemplate.update(
        "UPDATE users SET name = ?, email = ? WHERE id = ?",
        user.name,
        user.email,
        id
    )
}

fun deleteById(id: Long): Int {
    return jdbcTemplate.update(
        "DELETE FROM users WHERE id = ?",
        id
    )
}
```

UserController (1)

```
/* spring101.user.controller.UserController.kt (1) */
package spring101.user.controller

import spring101.user.model.User
import spring101.user.service.UserService
import org.springframework.http.HttpStatus
import org.springframework.web.bind.annotation.*
import org.springframework.stereotype.Controller
import org.springframework.ui.Model
import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.RequestMapping
import org.slf4j.LoggerFactory

@Controller
@RequestMapping("/api/users")
class UserController(private val userService: UserService) {

    private val logger = LoggerFactory.getLogger(UserController::class.java)

    init {
        logger.info("***** UserController initialized")
    }
}
```

```
/* spring101.user.controller.UserController.kt (2) */

@GetMapping
fun getAllUsers(model: Model): String {
    val users = userService.getAllUsers()
    val tableRows = users.joinToString("") {
        "<tr><td>${it.id}</td><td>${it.name}</td><td>${it.email}</td></tr>"
    }

    return """
    <!DOCTYPE html>
    <html lang="en">
    <head>
        <meta charset="UTF-8">
        <title>User List</title>
        <style>
            table { width: 100%; border-collapse: collapse; margin: 20px 0; }
            table, th, td { border: 1px solid black; }
            th, td { padding: 8px; text-align: left; }
            th { background-color: #f2f2f2; }
        </style>
    </head>
    <body>
        <h1>User List</h1>
        <table>
            <thead>
                <tr><th>ID</th><th>Name</th><th>Email</th></tr>
            </thead>
            <tbody>
                $tableRows
            </tbody>
        </table>
    </body>
    </html>
    """
}
```

UserController (2)

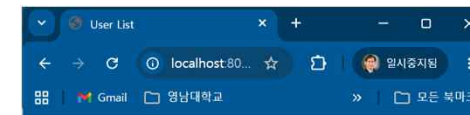
```
/* spring101.user.controller.UserController.kt (3) */
```

```
@GetMapping("/{id}")
fun getUserById(@PathVariable id: Long): User? {
    logger.info("Fetching user with ID: $id")
    return userService.getUserById(id)
}

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
fun createUser(@RequestBody user: User): User {
    logger.info("Creating user: $user")
    return userService.createUser(user)
}

@PutMapping("/{id}")
fun updateUser(@PathVariable id: Long, @RequestBody user: User):
User? {
    logger.info("Updating user with ID: $id")
    return userService.updateUser(id, user)
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
fun deleteUser(@PathVariable id: Long) {
    logger.info("Deleting user with ID: $id")
    userService.deleteUser(id)
}
}
```



User List

ID	Name	Email
1	Alice	alice@example.com
2	Bob	bob@example.com
3	Kim	kim@abc.com
4	Lee	lee@xyz.com
5	Park	park@yu.ac.kr
6	Choi	choi@sss.co.kr
7	Hong	hong@abc.ac.kr

application.properties, schema.sql, data.sql

application.properties

```
# H2 Database Configuration
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Execute schema.sql and data.sql
spring.sql.init.mode=always
spring.sql.init.schema-locations=classpath:schema.sql
spring.sql.init.data-locations=classpath:data.sql

# H2 Console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

management.endpoints.web.exposure.include=*
```



User List

ID	Name	Email
1	Alice	alice@example.com
2	Bob	bob@example.com
3	Kim	kim@abc.com
4	Lee	lee@xyz.com
5	Park	park@yu.ac.kr
6	Choi	choi@sss.co.kr
7	Hong	hong@abc.ac.kr

schema.sql

```
CREATE TABLE IF NOT EXISTS users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL
);
```

data.sql

```
INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com');
INSERT INTO users (name, email) VALUES ('Bob', 'bob@example.com');
INSERT INTO users (name, email) VALUES ('Kim', 'kim@abc.com');
INSERT INTO users (name, email) VALUES ('Lee', 'lee@xyz.com');
INSERT INTO users (name, email) VALUES ('Park', 'park@yu.ac.kr');
INSERT INTO users (name, email) VALUES ('Choi', 'choi@sss.co.kr');
INSERT INTO users (name, email) VALUES ('Hong', 'hong@abc.ac.kr');
```

Kotlin-SpringBoot 프로그램에 logging 기능 추가

◆ Spring Boot의 SLF4J(Simple Logging Facade for Java)

- LoggerFactory.getLogger()로 생성
- TRACE: 매우 상세한 정보를 기록하며, 디버깅이 필요한 경우에 사용.
- DEBUG: 개발 및 디버깅 시 유용한 정보를 기록.
- INFO: 애플리케이션의 실행 흐름이나 주요 상태 정보를 기록.
- WARN: 예외 상황이 발생할 가능성이 있거나 주의가 필요한 경우에 기록.
- ERROR: 오류가 발생했으며, 실행에 영향을 줄 수 있는 상황을 기록.

```
/* spring101.user.service.UserService.kt */
package spring101.user.service

import spring101.user.model.User
import spring101.user.repository.UserRepository
import org.springframework.stereotype.Service
import org.slf4j.Logger
import org.slf4j.LoggerFactory

@Service
class UserService(private val userRepository: UserRepository) {
    private val logger: Logger = LoggerFactory.getLogger(UserService::class.java)
    fun findUser(id: Long): User? {
        logger.info("Finding user with id: $id")
        val user : User? = userRepository.findById(id)/* user 검색 로직 */
        if (user == null) {
            logger.warn("User with id $id not found")
        }
        return user
    }
    ....
}
```

Application.kt

```
package spring101.user
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication
import org.slf4j.Logger
import org.slf4j.LoggerFactory
```

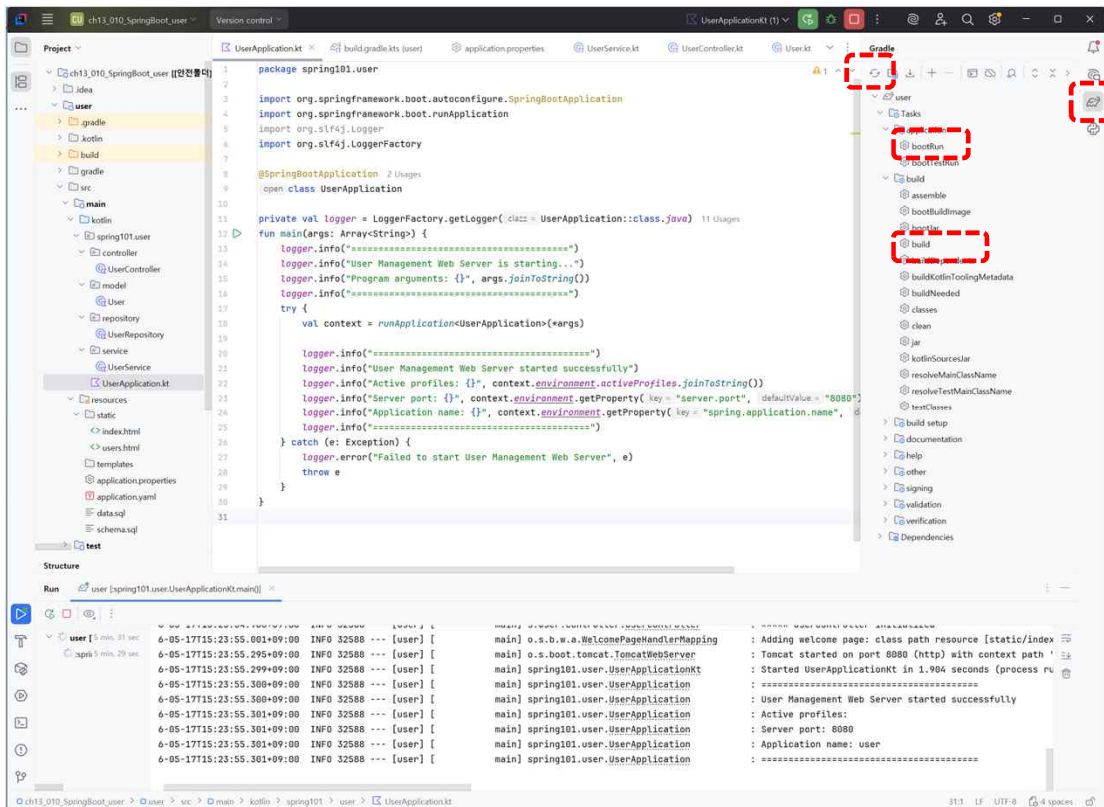
```
@SpringBootApplication
class UserApplication
```

```
private val logger = LoggerFactory.getLogger(UserApplication::class.java)
fun main(args: Array<String>) {
    logger.info("=====")
    logger.info("User Management Web Server is starting...")
    logger.info("Program arguments: {}", args.joinToString())
    logger.info("=====")
    try {
        val context = runApplication<UserApplication>(*args)

        logger.info("=====")
        logger.info("User Management Web Server started successfully")
        logger.info("Active profiles: {}", context.environment.activeProfiles.joinToString()))
        logger.info("Server port: {}",
            context.environment.getProperty("server.port", "8080"))
        logger.info("Application name: {}",
            context.environment.getProperty("spring.application.name", "unknown"))
        logger.info("=====")
    } catch (e: Exception) {
        logger.error("Failed to start User Management Web Server", e)
        throw e
    }
}
```

[illegible]

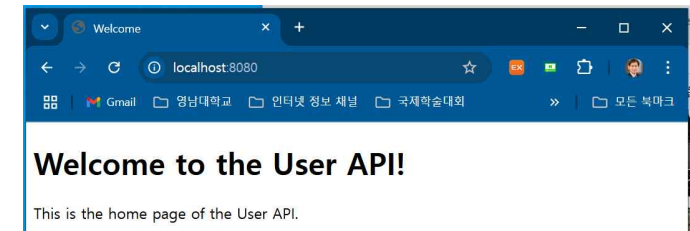
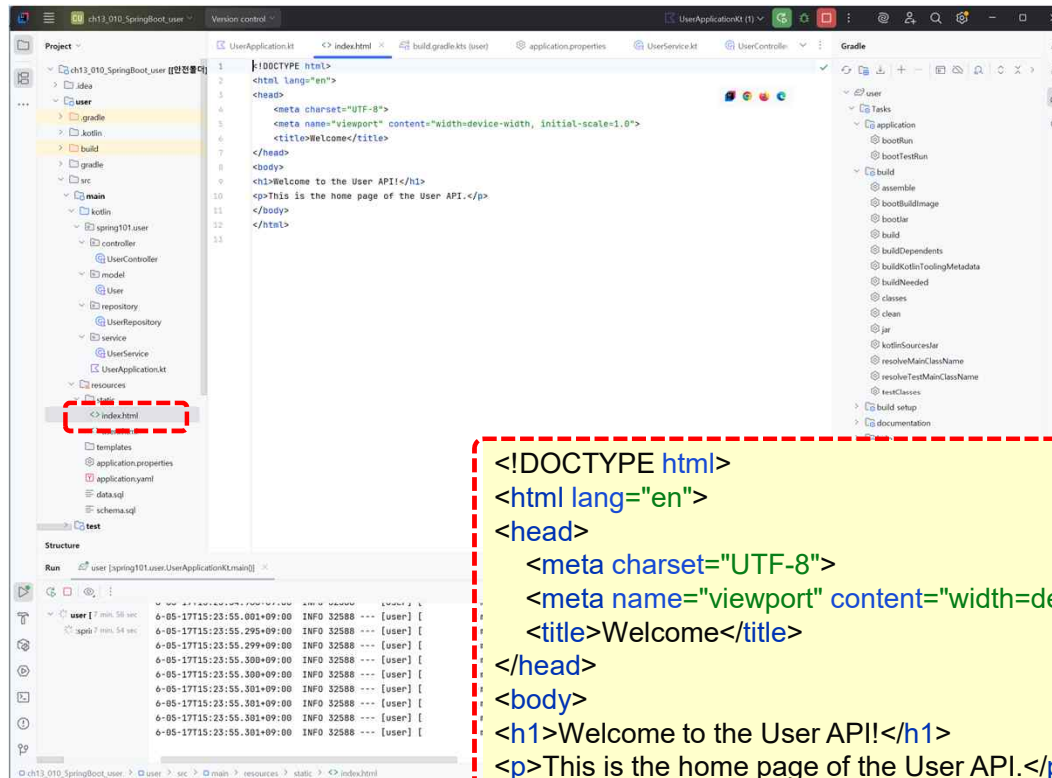
Build 및 실행, 웹서버 접속 및 결과 확인



User List

ID	Name	Email
1	Alice	alice@example.com
2	Bob	bob@example.com
3	Kim	kim@abc.com
4	Lee	lee@xyz.com
5	Park	park@yu.ac.kr
6	Choi	choi@sss.co.kr
7	Hong	hong@abc.ac.kr

resource directory에 index.html 추가, Web Server 접속



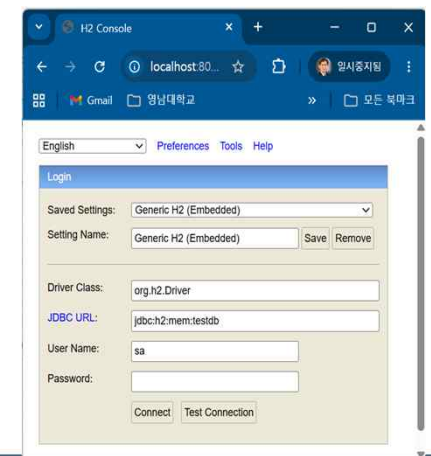
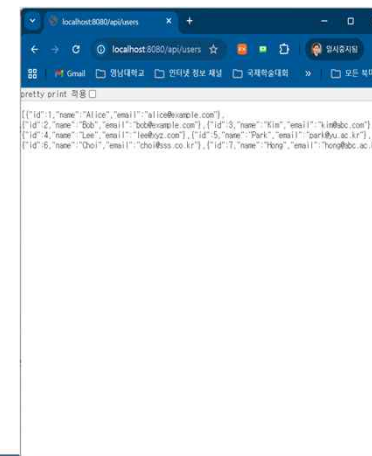
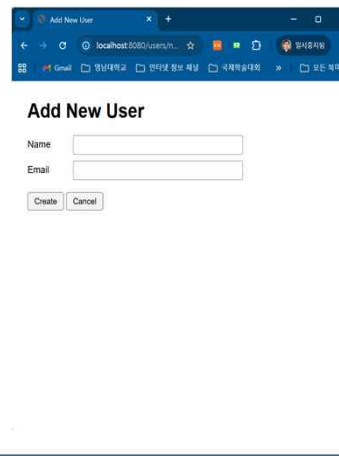
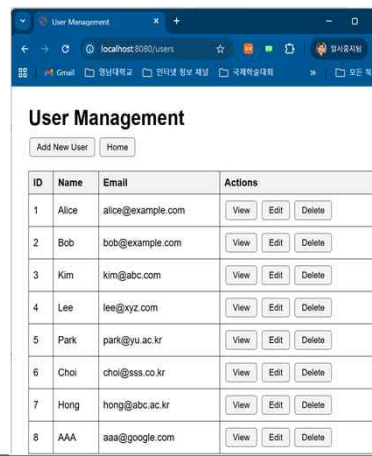
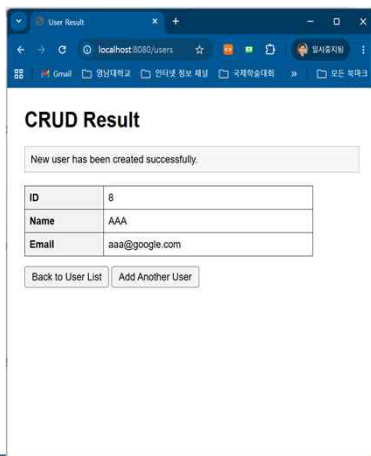
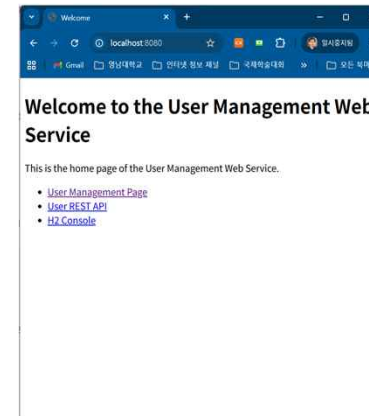
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Welcome</title>
</head>
<body>
  <h1>Welcome to the User API!</h1>
  <p>This is the home page of the User API.</p>
</body>
</html>
```

Kotlin-SpringBoot 기반 Web server에 CRUD 기능 구현

Kotlin-SpringBoot 기반 CRUD 예제

◆ CRUD (Create, Read, Update, Delete)

- 사용자 (user)의 생성, 검색, 갱신, 삭제 기능 지원
- H2 in-memory Database 사용
- Thymeleaf 기반 Web 페이지 사용
- Spring initializr를 사용하여 프로젝트 기본 파일 생성
- 초기화 데이터 설정 기능



Project Structure

◆ 전체 프로젝트 MVC 구조

- Model
- View
- Controller

◆ Model

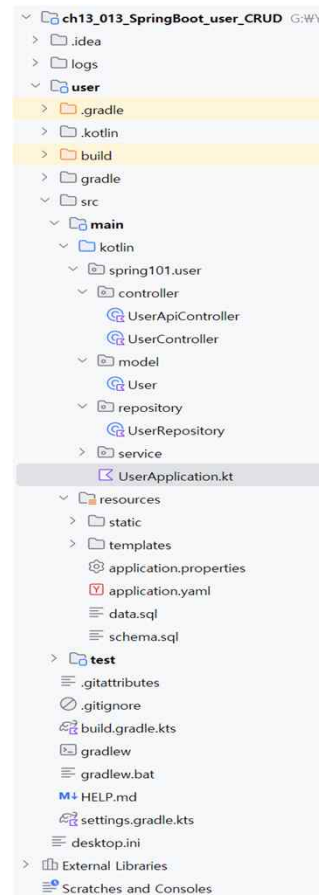
- User
- UserRepository
- UserService

◆ View

- users.html
- update-user.html

◆ Controller

- UserController



build.gradle.kts

```
plugins {
    kotlin("jvm") version "2.2.21"
    kotlin("plugin.spring") version "2.2.21"
    id("org.springframework.boot") version "4.0.6"
    id("io.spring.dependency-management") version "1.1.7"
}

group = "spring101"
version = "0.0.1-SNAPSHOT"

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.springframework.boot:spring-boot-starter-thymeleaf")
    implementation("org.springframework.boot:spring-boot-starter-webmvc")
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    implementation("tools.jackson.module:jackson-module-kotlin")
    compileOnly("org.projectlombok:lombok")
    annotationProcessor("org.projectlombok:lombok")
    testImplementation("org.springframework.boot:spring-boot-starter-thymeleaf-test")
    testImplementation("org.springframework.boot:spring-boot-starter-webmvc-test")
    testImplementation("org.jetbrains.kotlin:kotlin-test-junit5")
    testCompileOnly("org.projectlombok:lombok")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    testAnnotationProcessor("org.projectlombok:lombok")
    implementation("org.springframework.boot:spring-boot-starter-jdbc")
}
```

```
// Required in Spring Boot 4.x to enable the H2 web console auto-configuration
implementation("org.springframework.boot:spring-boot-h2console")
runtimeOnly("com.h2database:h2")
}

kotlin {
    compilerOptions {
        freeCompilerArgs.addAll("-Xjsr305=strict",
            "-Xannotation-default-target=param-property")
    }
}

tasks.withType<Test> {
    useJUnitPlatform()
}
```

Model – User, UserRepository, UserApiController

User.kt

```
/* spring101.user.model.User.kt */
package spring101.user.model

data class User(
    val id: Long = 0,
    val name: String = "",
    val email: String = ""
)
```

UserRepository.kt

```
/* spring101.user.repository.UserRepository.kt (1) */
package spring101.user.repository

import org.slf4j.LoggerFactory
import org.springframework.dao.EmptyResultDataAccessException
import org.springframework.jdbc.core.JdbcTemplate
import org.springframework.jdbc.core.RowMapper
import org.springframework.jdbc.support.GeneratedKeyHolder
import org.springframework.jdbc.support.KeyHolder
import org.springframework.stereotype.Repository
import spring101.user.model.User
import java.sql.PreparedStatement
import java.sql.Statement
```

/* spring101.user.repository.UserRepository.kt (2) */

```
@Repository
class UserRepository(
    private val jdbcTemplate: JdbcTemplate
) {
    private val logger =
        LoggerFactory.getLogger(UserRepository::class.java)

    private val rowMapper = RowMapper<User> { rs, _ ->
        User(
            id = rs.getLong("id"),
            name = rs.getString("name"),
            email = rs.getString("email")
        )
    }

    fun findAll(): List<User> {
        logger.debug("Executing SQL: SELECT id, name, email FROM users ORDER BY id")
        return jdbcTemplate.query(
            "SELECT id, name, email FROM users ORDER BY id",
            rowMapper
        )
    }
}
```

```
/* spring101.user.repository.UserRepository.kt (3) */
```

```
fun findById(id: Long): User? {
    logger.debug("Executing SQL: SELECT id, name, email FROM
        users WHERE id = ?, id={}", id)
    return try {
        jdbcTemplate.queryForObject(
            "SELECT id, name, email FROM users WHERE id = ?",
            rowMapper,
            id
        )
    } catch (e: EmptyResultDataAccessException) {
        logger.warn("No user found in database. id={}", id)
        null
    }
}

fun save(user: User): User {
    logger.debug("Executing SQL: INSERT INTO users (name, email)
        VALUES (?, ?)")

    val keyHolder: KeyHolder = GeneratedKeyHolder()
    jdbcTemplate.update({ connection ->
        val ps: PreparedStatement = connection.prepareStatement(
            "INSERT INTO users (name, email) VALUES (?, ?)",
            Statement.RETURN_GENERATED_KEYS
        )
        ps.setString(1, user.name)
        ps.setString(2, user.email)
        ps
    }, keyHolder)

    val generatedId = keyHolder.key?.toLong() ?: 0L
    logger.debug("Generated user id={}", generatedId)

    return user.copy(id = generatedId)
}
```

```
/* spring101.user.repository.UserRepository.kt (4) */
```

```
fun update(id: Long, user: User): Int {
    logger.debug("Executing SQL: UPDATE users SET name = ?,
        email = ? WHERE id = ?, id={}", id)
    return jdbcTemplate.update(
        "UPDATE users SET name = ?, email = ? WHERE id = ?",
        user.name,
        user.email,
        id
    )
}

fun deleteById(id: Long): Int {
    logger.debug("Executing SQL: DELETE FROM users
        WHERE id = ?, id={}", id)
    return jdbcTemplate.update(
        "DELETE FROM users WHERE id = ?",
        id
    )
}
```

Controller - UserApiController

```
/* spring101.user.controller.UserApiController.kt (1) */
package spring101.user.controller

import org.slf4j.LoggerFactory
import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity
import org.springframework.web.bind.annotation.DeleteMapping
import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.PathVariable
import org.springframework.web.bind.annotation.PostMapping
import org.springframework.web.bind.annotation.PutMapping
import org.springframework.web.bind.annotation.RequestBody
import org.springframework.web.bind.annotation.RequestMapping
import org.springframework.web.bind.annotation.RestController
import spring101.user.model.User
import spring101.user.service.UserService

@RestController
@RequestMapping("/api/users")
class UserApiController(
    private val userService: UserService
) {
    private val logger = LoggerFactory.getLogger(UserApiController::class.java)

    @GetMapping
    fun getAllUsers(): List<User> {
        logger.info("API request received: GET /api/users")
        return userService.getAllUsers()
    }

    @GetMapping("/{id}")
    fun getUserById(@PathVariable id: Long): ResponseEntity<User> {
        logger.info("API request received: GET /api/users/{id}", id)
        val user = userService.getUserById(id)
        return if (user == null) {
            ResponseEntity.notFound().build()
        } else {
            ResponseEntity.ok(user)
        }
    }
}
```

```
/* spring101.user.controller.UserApiController.kt (2) */

@PostMapping
fun createUser(@RequestBody user: User): ResponseEntity<User> {
    logger.info("API request received: POST /api/users")
    val savedUser = userService.createUser(user)
    return ResponseEntity.status(HttpStatus.CREATED).body(savedUser)
}

@PutMapping("/{id}")
fun updateUser(
    @PathVariable id: Long,
    @RequestBody user: User
): ResponseEntity<User> {
    logger.info("API request received: PUT /api/users/{id}", id)
    val updatedUser = userService.updateUser(id, user)
    return if (updatedUser == null) {
        ResponseEntity.notFound().build()
    } else {
        ResponseEntity.ok(updatedUser)
    }
}

@DeleteMapping("/{id}")
fun deleteUser(@PathVariable id: Long): ResponseEntity<Void> {
    logger.info("API request received: DELETE /api/users/{id}", id)
    val deleted = userService.deleteUser(id)
    return if (deleted) {
        ResponseEntity.noContent().build()
    } else {
        ResponseEntity.notFound().build()
    }
}
}
```

```
/* spring101.user.controller.UserApiController.kt (1) */
package spring101.user.controller

import org.slf4j.LoggerFactory
import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity
import org.springframework.web.bind.annotation.DeleteMapping
import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.PathVariable
import org.springframework.web.bind.annotation.PostMapping
import org.springframework.web.bind.annotation.PutMapping
import org.springframework.web.bind.annotation.RequestBody
import org.springframework.web.bind.annotation.RequestMapping
import org.springframework.web.bind.annotation.RestController
import spring101.user.model.User
import spring101.user.service.UserService

@RestController
@RequestMapping("/api/users")
class UserApiController(
    private val userService: UserService
) {
    private val logger = LoggerFactory.getLogger(UserApiController::class.java)

    @GetMapping
    fun getAllUsers(): List<User> {
        logger.info("API request received: GET /api/users")
        return userService.getAllUsers()
    }

    @GetMapping("/{id}")
    fun getUserById(@PathVariable id: Long): ResponseEntity<User> {
        logger.info("API request received: GET /api/users/{id}", id)
        val user = userService.getUserById(id)
        return if (user == null) {
            ResponseEntity.notFound().build()
        } else {
            ResponseEntity.ok(user)
        }
    }
}
```

```
/* spring101.user.controller.UserApiController.kt (2) */

@PostMapping
fun createUser(@RequestBody user: User): ResponseEntity<User> {
    logger.info("API request received: POST /api/users")
    val savedUser = userService.createUser(user)
    return ResponseEntity.status(HttpStatus.CREATED).body(savedUser)
}

@PutMapping("/{id}")
fun updateUser(
    @PathVariable id: Long,
    @RequestBody user: User
): ResponseEntity<User> {
    logger.info("API request received: PUT /api/users/{id}", id)
    val updatedUser = userService.updateUser(id, user)
    return if (updatedUser == null) {
        ResponseEntity.notFound().build()
    } else {
        ResponseEntity.ok(updatedUser)
    }
}

@DeleteMapping("/{id}")
fun deleteUser(@PathVariable id: Long): ResponseEntity<Void> {
    logger.info("API request received: DELETE /api/users/{id}", id)
    val deleted = userService.deleteUser(id)
    return if (deleted) {
        ResponseEntity.noContent().build()
    } else {
        ResponseEntity.notFound().build()
    }
}
}
```

View – users.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>User List</title>
  <style>
    table {
      width: 100%;
      border-collapse: collapse;
      margin: 20px 0;
    }
    table, th, td {
      border: 1px solid black;
    }
    th, td {
      padding: 8px;
      text-align: left;
    }
    th {
      background-color: #f2f2f2;
    }
  </style>
</head>

```

```

<body>
<h1>User List</h1>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Email</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="user : ${users}">
      <td th:text="${user.id}"></td>
      <td th:text="${user.name}"></td>
      <td th:text="${user.email}"></td>
    </tr>
  </tbody>
</table>
</body>
</html>

```

View - index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Welcome</title>
</head>
<body>
<h1>Welcome to the User Management Web Service</h1>
<p>This is the home page of the User Management Web Service.</p>
<ul>
  <li><a href="/users">User Management Page</a></li>
  <li><a href="/api/users">User REST API</a></li>
  <li><a href="/h2-console">H2 Console</a></li>
</ul>
</body>
</html>
```

data.sql, schema.sql

schema.sql

```
CREATE TABLE IF NOT EXISTS users (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL  
);
```

data.sql

```
INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com');  
INSERT INTO users (name, email) VALUES ('Bob', 'bob@example.com');  
INSERT INTO users (name, email) VALUES ('Kim', 'kim@abc.com');  
INSERT INTO users (name, email) VALUES ('Lee', 'lee@xyz.com');  
INSERT INTO users (name, email) VALUES ('Park', 'park@yu.ac.kr');  
INSERT INTO users (name, email) VALUES ('Choi', 'choi@sss.co.kr');  
INSERT INTO users (name, email) VALUES ('Hong', 'hong@abc.ac.kr');
```

UserApplication.kt

```

/* spring101.user.UserApplication.kt */

package spring101.user

import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication
import org.slf4j.Logger
import org.slf4j.LoggerFactory

@SpringBootApplication
class UserApplication

private val logger = LoggerFactory.getLogger(UserApplication::class.java)
fun main(args: Array<String>) {
    logger.info("=====")
    logger.info("User Management Web Server is starting...")
    logger.info("Program arguments: {}", args.joinToString())
    logger.info("=====")
    try {
        val context = runApplication<UserApplication>(*args)

        logger.info("=====")
        logger.info("User Management Web Server started successfully")
        logger.info("Active profiles: {}", context.environment.activeProfiles.joinToString())
        logger.info("Server port: {}", context.environment.getProperty("server.port", "8080"))
        logger.info("Application name: {}",
            context.environment.getProperty("spring.application.name", "unknown"))
        logger.info("=====")
    } catch (e: Exception) {
        logger.error("Failed to start User Management Web Server", e)
        throw e
    }
}

```

application.properties

```
# H2 Database Configuration
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Execute schema.sql and data.sql
spring.sql.init.mode=always
spring.sql.init.schema-locations=classpath:schema.sql
spring.sql.init.data-locations=classpath:data.sql

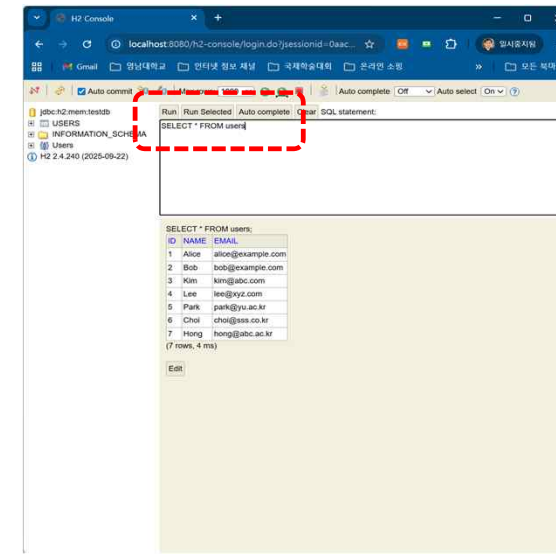
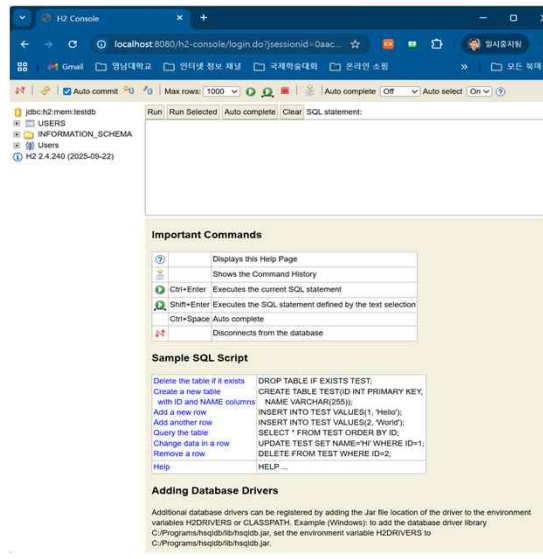
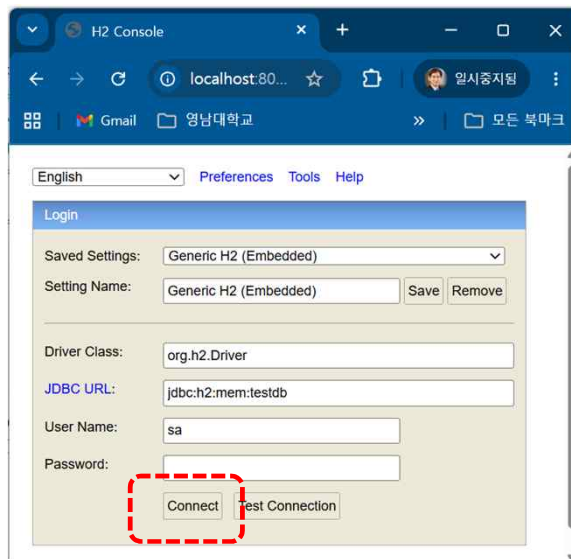
# H2 Console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

management.endpoints.web.exposure.include=*
# Logging Configuration
logging.level.root=INFO
logging.level.spring101.user=DEBUG
logging.level.org.springframework.jdbc.core.JdbcTemplate=DEBUG
logging.level.org.springframework.jdbc.core.StatementCreatorUtils=TRACE
logging.file.name=logs/user-management.log
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
```

H2 Database 접속 및 확인

◆ H2 in-memory Database

- <http://localhost:8080/h2-console>
- DBC URL: jdbc:h2:mem:testdb
- Username: sa
- Password:



Spring WebFlux, WebClient

Spring WebFlux

◆ Spring WebFlux, WebClient

- <https://docs.spring.io/spring-framework/reference/web/webflux.html>
- <https://docs.spring.io/spring-framework/reference/web/webflux-webclient.html>
- Spring WebFlux 는 Spring 5.0에서 도입된 비동기, 논블로킹(non-blocking) 웹 프레임워크
- MVC 대비 사용률은 낮으나, 고성능, 고동시성이 필요한 특정 마이크로서비스나 게이트웨이 영역에서 채택이 빠르게 증가하고 있음
- Non-Blocking I/O 기반의 Event Loop(이벤트 루프) 모델을 사용하여 적은 수의 스레드로 대규모 동시 요청을 효율적으로 처리함
- I/O 작업이 많고 대기 시간이 긴 환경(예: 다수의 외부 API 호출), 높은 처리량(Throughput)이 요구되는 API 게이트웨이, 실시간 데이터 스트리밍(SSE), 채팅 서버 등 고동시성 환경에 적합함
- WebFlux는 주로 리액티브 프로그래밍 모델 (reactive programming model)을 사용하여 고성능 비동기 애플리케이션을 구축하는 데 중점을 둠; 리액티브 프로그래밍의 개념(Mono, Flux) 자체가 생소하여 MVC 대비 개발 난이도가 높고 코드의 흐름이 복잡해질 수 있음
- Kotlin과 함께 사용할 수 있는 리액티브 프로그래밍 스타일을 지원하여 효율적이고 확장 가능한 애플리케이션을 만들 수 있게 해 줌
- Spring WebFlux와 함께 사용되는 Spring WebClient는 Spring 5.0에서 도입된 비동기식 및 반응형 HTTP 클라이언트
- Spring WebClient는 기존의 동기방식의 RestTemplate을 대체하며, 비동기 및 반응형 방식으로 설계되어 더 높은 성능과 유연성을 제공하며, 이는 특히 비동기 방식이 중요한 마이크로서비스 아키텍처나 고성능 시스템에서 유용함

Spring MVC와 Spring WebFlux의 비교

항목	Spring MVC	Spring WebFlux
처리 방식	동기, 블로킹 I/O	비동기, 논블로킹 I/O
프로그래밍 모델	전통적인 Spring MVC (Servlet API 기반)	리액티브 프로그래밍 (리액티브 스트림 기반)
서버 지원	주로 Tomcat, Jetty 등 전통적인 블로킹 서버	Netty, Undertow 등 논블로킹 서버 (Tomcat도 지원 가능)
비동기 처리	콜백 지옥, 동시성 처리가 복잡할 수 있음	리액터 API 또는 코루틴을 사용한 간결한 비동기 처리 가능
데이터 반환 형식	동기적으로 데이터를 반환 (예: List, Object)	Mono와 Flux로 리액티브 스트림 형식의 비동기 데이터 반환
적용 사례	동기적이고 전통적인 웹 애플리케이션, 트랜잭션 처리 중심	I/O 바운드, 높은 동시성이 요구되는 애플리케이션에 적합
사용 기준 및 상황	대다수의 경우, Spring Boot + MVC로 충분한 성능을 발휘할 수 있음. 대부분의 일반적인 비즈니스 애플리케이션 개발 시. 비즈니스 로직의 복잡도가 높고, 코딩의 직관성 및 유지 보수 가 중요할 때. 전통적인 RDB(JDBC/JPA)를 사용하고, Blocking 방식의 라이브러리가 주를 이룰 때.	WebFlux는 그 복잡성 때문에 성능 최적화가 필수적인 특정 영역 에 한정하여 선택하는 전략적 도구로 활용되는 것이 일반적임. 초당 수만 건 이상의 요청 등 대규모 동시 접속 처리가 필수일 때. 외부 시스템과의 잡은 비동기 통신 (WebClient) 이 핵심일 때. 시스템의 I/O 대기 시간이 길어 스레드 효율성을 극대화해야 할 때.

Reactive Programming Model

◆ Reactive Programming Model

- Reactive Programming은 데이터 스트림과 변경 사항을 비동기적으로 처리하는 프로그래밍 패러다임
- WebFlux는 이러한 개념을 웹 애플리케이션에 적용하여 고성능, 확장성이 뛰어난 시스템을 구축할 수 있게 함

◆ Reactive Programming의 주요 개념:

- 데이터 스트림: 데이터는 시간의 흐름에 따라 발생하며, 이벤트처럼 스트림으로 처리됨
- 비동기 처리: 요청과 응답의 처리가 비동기로 이루어져 블로킹 없는 시스템을 만듦
- 백프레셔(Backpressure): 데이터 스트림의 생산자와 소비자 간 속도 차이를 제어하여 소비자가 처리 가능한 만큼만 데이터를 전달함 (통신 프로토콜의 **흐름제어 (flow control)** 기능 제공)

Reactive Streams와 WebFlux

◆ Spring WebFlux와 Reactor

- Spring WebFlux는 Reactive Streams 스펙을 준수하며, Reactor라는 구현체를 사용함

◆ Reactive Streams는 네 가지 주요 인터페이스:

- Publisher: 데이터의 생산자를 정의하며, WebFlux에서 HTTP 요청과 응답은 Publisher로 표현됨
- Subscriber: 데이터의 소비자 (가입자)를 정의함
- Subscription: Publisher와 Subscriber 간의 계약을 정의하며, 데이터 소비 속도 제어(Backpressure)를 지원함
- Processor: Publisher와 Subscriber의 역할을 동시에 수행함

◆ Spring WebFlux의 Reactor는 두 가지 주요 Publisher를 사용

- Mono<T>: 단일값을 비동기로 제공하는 Publisher
- Flux<T>: 다수(0개 이상)의 데이터를 스트림으로 제공하는 Publisher

WebFlux의 Non-blocking I/O

◆ WebFlux의 Non-blocking I/O

- Non-blocking I/O는 I/O 작업 (읽기, 쓰기, 네트워크 요청 등)이 완료될 때 까지 스레드가 기다리지 않고 다른 작업을 계속 수행할 수 있게 하는 방식을 의미하며, 요청 작업이 완료되면 callback이나 event로 해당 결과를 처리할 수 있게 하는 방식임
- Spring WebFlux는 전통적인 Spring MVC와는 달리 non-blocking I/O 모델을 활용하여 스레드 효율성을 극대화할 수 있게 하며, 적은 수의 스레드로도 많은 요청을 처리할 수 있게 함; WebFlux는 **적은 리소스로 많은 동시 접속을 처리하는** 것이 최우선 목표일 때 유용하며, 특히 마이크로서비스 아키텍처에서 다른 서비스와의 통신(API Gateway)에 **WebClient**를 사용하여 논블로킹 통신을 구현할 때 큰 장점을 가짐
- WebFlux는 이벤트 기반으로 동작하며, 요청/응답 처리가 비동기적으로 이루어지고, 작업이 완료되면 callback을 통하여 결과를 처리함; WebFlux는 Reactive Streams API를 사용하여 데이터를 스트림 형태로 처리할 수 있게 하며, non-blocking 방식으로 데이터 흐름을 제어함
- Spring WebFlux는 기본적으로 Netty를 기반으로 동작하는데, Netty는 고성능 비동기 네트워크 프레임워크로 non-blocking I/O를 효율적으로 지원함
- WebFlux는 요청을 처리할 때 sublet 스레드 풀을 사용하지 않고, Netty와 같은 이벤트 루프 (event loop)를 통해 요청을 비동기적으로 처리함. 특히 데이터베이스 호출, 외부 API 요청 등 시간이 오래 걸리는 작업은 blocking 없이 비동기적으로 실행됨. 호출된 작업이 완료되면 callback에 의하여 결과를 처리함.
- Non-blocking I/O 방식을 사용하는 WebFlux에서는 작업이 완료되면 요청 응답을 stream 형태로 client에게 반환함.

Spring WebFlux의 주요 구성 요소

◆ Handler와 Router

- Spring WebFlux는 전통적인 @Controller 기반의 선언적 프로그래밍 방식 외에도 함수형 프로그래밍 스타일을 지원
- Handler: HTTP 요청을 처리하는 함수.
- Router: HTTP 경로와 Handler를 연결.

```
val route = RouterFunctions.route()  
    .GET("/hello") { ServerResponse.ok().bodyValue("Hello, WebFlux!") }  
    .build()
```

- WebClient: RestTemplate의 비동기 버전으로, 외부 API 호출 시 non-blocking 방식으로 작동함

```
val client = WebClient.create()  
client.get()  
    .uri("https://api.example.com")  
    .retrieve()  
    .bodyToMono(String::class.java)  
    .subscribe { println(it) }
```

WebFlux의 함수형 프로그래밍 지원

◆ WebFlux 프레임워크에서의 Kotlin Annotation

어노테이션	적용 대상	설명
@SpringBootApplication	클래스	Spring Boot 애플리케이션의 진입점을 나타내는 어노테이션. @Configuration, @EnableAutoConfiguration, @ComponentScan을 포함하여 Spring 설정 및 컴포넌트 스캔을 자동으로 설정함.
@Bean	메서드	Spring 애플리케이션 컨텍스트에서 관리하는 객체(빈)를 정의하는 메서드에 사용됨. 이 메서드가 반환하는 객체는 Spring에 의해 빈으로 관리됨.
@Controller	클래스	웹 요청을 처리하는 클래스에 사용되는 어노테이션. WebFlux나 MVC에서 사용되며, URL 요청을 매핑하여 적절한 응답을 처리하는 역할을 함.
@RestController	클래스	@Controller와 유사하지만, 각 메서드가 자동으로 @ResponseBody가 적용되어 JSON 등의 데이터 형태로 응답을 반환하도록 설계된 컨트롤러임.
@RequestMapping	클래스, 메서드	특정 URL 패턴과 HTTP 메서드(GET, POST 등)를 특정 클래스나 메서드에 매핑할 때 사용됨. 주로 컨트롤러 클래스나 메서드에서 웹 요청을 처리할 때 사용함.
@GetMapping	메서드	HTTP GET 요청을 처리하는 메서드에 사용됨. 주로 리소스를 조회하거나 특정 데이터를 얻는 작업에 사용됨.
@PostMapping	메서드	HTTP POST 요청을 처리하는 메서드에 사용됨. 주로 데이터를 생성하거나 서버에 새로운 정보를 전송할 때 사용됨.
@PutMapping	메서드	HTTP PUT 요청을 처리하는 메서드에 사용됨. 리소스의 업데이트 작업에 주로 사용됨.
@DeleteMapping	메서드	HTTP DELETE 요청을 처리하는 메서드에 사용됨. 주로 리소스를 삭제할 때 사용됨.
@PathVariable	메서드 파라미터	URL 경로의 변수를 메서드의 파라미터에 매핑할 때 사용됨. 예를 들어, /users/{id}와 같이 URL에서 특정 값을 받아 처리할 때 사용됨.
@RequestParam	메서드 파라미터	HTTP 요청의 쿼리 파라미터를 메서드의 파라미터에 매핑할 때 사용됨. 예를 들어, /users?name=John에서 name파라미터를 받아 처리할 때 사용됨.
@RequestBody	메서드 파라미터	HTTP 요청 본문(body)의 데이터를 메서드의 파라미터로 매핑할 때 사용됨. 주로 JSON 데이터를 처리할 때 사용됨.
@ResponseBody	메서드	메서드에서 반환하는 데이터를 HTTP 응답 본문으로 직렬화(Serialize)할 때 사용됨. 주로 JSON 형식으로 변환하여 클라이언트에 응답을 반환함.
@Component	클래스	Spring 컨테이너에서 관리되는 일반적인 빈을 정의할 때 사용되는 어노테이션임. 이 어노테이션을 사용하면 Spring에서 해당 클래스를 자동으로 감지하여 빈으로 등록함.
@Service	클래스	서비스 레이어에 위치한 클래스에 사용됨. 비즈니스 로직을 처리하는 클래스를 나타내며, @Component와 같은 역할을 하지만 비즈니스 로직 계층에서 사용되는 빈임을 나타냄.
@Repository	클래스	데이터베이스와의 상호작용을 처리하는 클래스에 사용됨. 주로 데이터 접근 계층(DAO)에서 사용되며, Spring이 예외 처리를 자동으로 변환해 줌.
@Configuration	클래스	Spring에서 설정 클래스임을 나타냄. 이 어노테이션이 붙은 클래스는 Spring Bean을 정의하는 메서드를 포함하며, 주로 어플리케이션 설정을 담당함.
@EnableWebFlux	클래스	Spring WebFlux를 활성화하고, WebFlux 설정을 자동으로 구성함. Reactive 프로그래밍을 위한 비동기 웹 애플리케이션을 개발할 때 사용됨.
@EnableReactiveMongoRepositories	클래스	MongoDB의 리액티브 리포지토리를 활성화할 때 사용됨. 주로 리액티브 데이터베이스와 함께 사용할 때 적용됨.

WebFlux의 함수형 프로그래밍 지원

◆ WebFlux의 함수형 프로그래밍 지원

- Spring WebFlux는 함수형 프로그래밍 스타일로 애플리케이션을 개발할 수 있도록 설계되어 있으며, 이는 전통적인 애노테이션 기반 프로그래밍 방식과 비교하여 더 간결하고 유연한 코드를 작성할 수 있게 함.
- 예를 들어 Spring WebFlux는 HTTP 요청과 핸들러를 함수형 스타일로 정의할 수 있는 `router{}`를 제공하며, DSL (domain specific language)을 사용하여 라우팅 규칙을 정의할 수 있게 하여 코드를 더욱 간결하고 명확하게 작성할 수 있게 하며, 함수형 프로그래밍 스타일을 선호하는 개발자에게 적합하게 함
- Spring WebFlux의 함수형 프로그래밍 스타일에서는 핸들러를 별도의 클래스 또는 함수로 정의할 수 있으며, 이렇게 별도의 클래스로 구현된 핸들러를 라우터에 연결하여 사용할 수 있게 함

WebFlux의 함수형 프로그래밍 지원

◆ WebFlux의 Handler class와 Router로의 연결 예제

```
class MyHandler {  
  
    fun greet(request: ServerRequest): Mono<ServerResponse> {  
        return ServerResponse.ok().bodyValue("Hello, WebFlux!")  
    }  
  
    fun echo(request: ServerRequest): Mono<ServerResponse> {  
        return request.bodyToMono(String::class.java)  
            .flatMap { message ->  
                ServerResponse.ok().bodyValue("You said: $message")  
            }  
    }  
}  
  
val myHandler = MyHandler()  
  
val route = router {  
    GET("/greet", myHandler::greet)  
    POST("/echo", myHandler::echo)  
}
```

WebFlux의 Kotlin Coroutine 통합

◆ WebFlux의 Kotlin Coroutine 통합

- Spring WebFlux는 Kotlin의 Coroutine과 자연스럽게 통합되어 비동기 프로그래밍을 더욱 직관적으로 작성할 수 있게 함
- Kotlin Coroutine은 가벼운 비동기 프로그래밍 모델이며, suspend 키워드를 사용하여 non-blocking 코드를 동기 코드 처럼 작성할 수 있게 해 줌.
- Coroutine은 경량 스레드이며, 실제 스레드가 아닌 가벼운 작업 단위로 수천개의 코루틴을 동시에 실행 가능하게 함
- Coroutine은 비동기 작업을 동기 방식 처럼 작성할 수 있게 하며, 자원을 효율적으로 관리하며 오류를 처리할 수 있게하는 장점을 제공한다.
- Kotlin Coroutine은 WebFlux에서 Reactive Stream 기반의 Mono와 Flux를 효율적으로 처리하기 위한 대안적인 접근 방식을 제공함.
- 이는 kotlinx-coroutine-reactor 라이브러리를 사용하여 구현할 수 있으며, Mono와 Flux를 Coroutine으로 변환할 수 있는 확장 함수를 사용할 수 있음

WebFlux의 Kotlin Coroutine 통합

◆ suspend 함수를 사용하여 비동기 작업 수행

- Spring WebFlux의 핸들러 메서드를 suspend 함수로 정의하여 비동기 작업을 처리할 수 있음
- 아래 예제에서 suspend fun으로 정의된 greet() 메서드는 non-blocking 방식으로 동작하게 되며, service.getGreeting() 호출은 coroutine 안에서 비동기적으로 실행됨

```
@RestController
@RequestMapping("/api")
class CoroutineController(private val service: CoroutineService) {
    @GetMapping("/greet")
    suspend fun greet(): String {
        return service.getGreeting() // 논블로킹 호출
    }
}
```

WebFlux의 Kotlin Coroutine 통합

◆ Mono 및 Flux를 Coroutine으로 변환

- Kotlin Coroutines는 Mono와 Flux를 쉽게 변환할 수 있는 확장함수를 제공함
- Mono.toSuspendFunction()은 Mono를 suspend 함수로 변환하며, Flux.toFlow()는 Flux를 Kotlin의 Flow로 변환하여 줌
- 다음 예제는 suspend fun으로 정의되어 coroutine으로 실행되는 fetchData() 함수의 결과를 Mono로 반환하는 예제를 보여줌

```
suspend fun fetchData(): String {  
    val mono = Mono.just("Hello, WebFlux with Coroutines!")  
    return mono.awaitSingle() // Mono를 suspend 함수로 변환  
}
```

WebFlux의 Kotlin Coroutine 통합

◆ Coroutine 기반의 서비스

- WebFlux 서비스를 구현할 때, service 계층에서도 coroutine을 사용할 수 있음
- 다음 예에서는 @Service 애노테이션으로 설정된 서비스 계층 클래스 CoroutineService의 메서드 getGreeting()과 getDataFromDatabase()를 suspend fun으로 정의하여 coroutine으로 실행되도록 설정하는 예를 보여줌.

```
@Service
class CoroutineService {
    suspend fun getGreeting(): String {
        delay(1000) // 비동기적으로 1초 대기
        return "Hello from Coroutine Service!"
    }

    suspend fun getDataFromDatabase(): List<String> {
        return listOf("Item1", "Item2", "Item3")
    }
}
```

WebFlux의 Kotlin Coroutine 통합

◆ Kotlin 클래스/함수형으로 구현된 CoroutineHandler를 Router에 통합

- 아래 예제 프로그램에서 class CoroutineHandler는 suspend fun으로 설정된 메서드 greet()와 getItems()를 coroutine으로 실행시키도록 구현되어 있으며, coRouter 생성에서 GET() 라우팅 기능 구현에서 class CoroutineHandler의 메서드를 각각 연결시켜 사용할 수 있도록 구현하고 있음

```
class CoroutineHandler {
    suspend fun greet(request: ServerRequest): ServerResponse {
        return ServerResponse.ok().bodyValue("Hello, Coroutine!")
    }
    suspend fun getItems(request: ServerRequest): ServerResponse {
        val items = listOf("Item1", "Item2", "Item3")
        return ServerResponse.ok().bodyValue(items)
    }
}

val coroutineHandler = CoroutineHandler()
val route = coRouter {
    GET("/greet", coroutineHandler::greet)
    GET("/items", coroutineHandler::getItems)
}
```

WebFlux의 Kotlin Coroutine 통합

◆ Kotlin Coroutine을 활용한 비동기 데이터베이스 호출

- Spring Data R2DBC (Reactive Relational Database Connectivity)를 사용하면 non-blocking 데이터베이스 작업을 coroutine으로 처리할 수 있음
- 다음 예제에서는 @Repository 애노테이션을 사용하여 데이터베이스와의 상호 작용을 처리하는 인터페이스 UserRepository를 선언하고, 이를 @Service 애노테이션으로 설정된 UserService에서 사용하는 예를 보여줌
- suspend fun으로 설정된 getUserById() 메서드는 내부에서 userRepository.findById()를 사용하며, coroutine 구조의 비동기식 데이터베이스 호출이 가능하도록 구현하고 있음

```
@Repository
interface UserRepository : CoroutineCrudRepository<User, Long>

@Service
class UserService(private val userRepository: UserRepository) {
    suspend fun getUserById(id: Long): User? {
        return userRepository.findById(id)
    }
}
```

WebSocket

◆ WebSocket

- WebSocket은 양방향 통신을 지원하며, 실시간 애플리케이션 개발에 적합함
- WebSocket의 주요 메서드의 함수원형, 설명 및 사용 예시:

메서드	함수 원형	설명	사용 예시
send	<code>Mono<Void> send(Publisher<WebSocketMessage> messages)</code>	클라이언트로 메시지를 전송	<code>kotlin session.send(Flux.just(session.textMessage("Hello, WebSocket!")))</code>
receive	<code>Flux<WebSocketMessage> receive()</code>	클라이언트로부터 받은 메시지를 스트림 형태로 제공	<code>kotlin session.receive().map { it.payloadAsText }.doOnNext { println(it) }.subscribe()</code>
text Message	<code>WebSocketMessage textMessage(String payload)</code>	텍스트 메시지를 생성	<code>kotlin session.textMessage("This is a text message")</code>
binary Message	<code>WebSocketMessage binaryMessage(Consumer<DataBuffer> payload)</code>	바이너리 메시지를 생성	<code>kotlin session.binaryMessage { buffer -> buffer.write(byteArrayOf(1, 2, 3)) }</code>
ping Message	<code>WebSocketMessage pingMessage(Consumer<DataBuffer> payload)</code>	클라이언트로 핑 메시지를 전송	<code>kotlin session.pingMessage { buffer -> buffer.write("Ping".toByteArray()) }</code>
close	<code>Mono<Void> close(CloseStatus status)</code>	지정된 상태 코드로 WebSocket 연결을 닫음	<code>kotlin session.close(CloseStatus.NORMAL)</code>
close Status	<code>CloseStatus closeStatus()</code>	연결이 닫힐 때의 상태를 가져옴	<code>kotlin val status = session.closeStatus()</code>
buffer Factory	<code>DataBufferFactory bufferFactory()</code>	데이터 버퍼를 생성할 때 사용하는 팩토리를 반환	<code>kotlin val buffer = session.bufferFactory().wrap("Data".toByteArray())</code>
isOpen	<code>boolean isOpen()</code>	WebSocket 연결이 열려 있는지 여부를 확인	<code>kotlin if (session.isOpen) { println("Connection is open") }</code>

WebSocket

◆ WebSocket 설정 및 처리 예제

```
import org.springframework.web.reactive.socket.WebSocketHandler
import org.springframework.web.reactive.socket.WebSocketSession
import reactor.core.publisher.Mono
class EchoWebSocketHandler : WebSocketHandler {
    override fun handle(session: WebSocketSession): Mono<Void> {
        return session.send( session.receive()
            .map { msg -> session.textMessage("
                Echo: ${msg.payloadAsText}") }
        )
    }
}
```

```
import org.springframework.context.annotation.Bean
import org.springframework.context.annotation.Configuration
import org.springframework.web.reactive.handler.WebSocketHandlerMapping
import org.springframework.web.reactive.socket.WebSocketHandler
import org.springframework.web.reactive.socket.server.support.WebSocketHandlerAdapter
@Configuration
class WebSocketConfig {
    @Bean // WebSocket 핸들러 매핑
    fun webSocketHandlerMapping(echoWebSocketHandler: WebSocketHandler):
        WebSocketHandlerMapping {
        val mapping = WebSocketHandlerMapping()
        mapping.order = 1 // 핸들러 매핑 우선순위
        mapping.urlMap = mapOf("/ws/echo" to echoWebSocketHandler )
        // "/ws/echo" 경로에 echoWebSocketHandler 매핑
        return mapping
    }
    @Bean // WebSocket 핸들러 어댑터
    fun webSocketHandlerAdapter(): WebSocketHandlerAdapter {
        return WebSocketHandlerAdapter()
    }
    @Bean // Echo WebSocket 핸들러 등록
    fun echoWebSocketHandler(): WebSocketHandler {
        return EchoWebSocketHandler()
        // 위에서 정의한 EchoWebSocketHandler를 사용
    }
}
```

WebFlux와 R2DBC 기반 Database 연동

◆ Spring Data R2DBC (Reactive Relational Database Connectivity)

- Spring Data R2DBC (Reactive Relational Database Connectivity)는 관계형 데이터베이스를 비동기적으로 호출할 수 있는 기능을 제공하며, JDBC와 달리 블로킹이 없는 non-blocking 방식으로 데이터베이스 작업을 처리할 수 있게 함
- R2DBC는 Flux 및 Mono를 사용하여 데이터를 스트림 형태로 처리할 수 있게 하며, PostgreSQL, MySQL, H2 등 주요 데이터베이스와 연동하여 사용할 수 있게 함
- Spring Data R2DBC는 Coroutine과 통합하여 Mono와 Flux 대신 suspend 함수와 Flow를 사용할 수 있으며, R2DBC에서 반환되는 Reactive Streams를 Coroutine 친화적인 방식으로 변환하여 사용할 수 있게 함
- R2DBC 사용을 위한 Dependencies 설정, YAML 파일에 연결정보 추가

```
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-data-r2dbc")
    implementation("org.springframework.boot:spring-boot-starter-webflux")
    implementation("io.r2dbc:r2dbc-postgresql") // PostgreSQL 드라이버 예시
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor")
}
```

```
spring:
  r2dbc:
    url: r2dbc:postgresql://localhost:5432/mydb
    username: myuser
    password: mypassword
```

WebFlux와 R2DBC 기반 Database 연동

◆ R2DBC를 사용하기 위한 Entity 정의

```
import org.springframework.data.annotation.Id
import org.springframework.data.relational.core.mapping.Table
@Table("users")
data class User(
    @Id val id: Long? = null,
    val name: String,
    val email: String
)
```

◆ R2DBC를 사용하기 위한 Repository 정의

```
import org.springframework.data.repository.kotlin.CoroutineCrudRepository
interface UserRepository : CoroutineCrudRepository<User, Long> {
    suspend fun findByName(name: String): User?
}
```

WebFlux와 R2DBC 기반 Database 연동

◆ R2DBC 기반 데이터베이스 연동 기능이 포함된 서비스 계층 예제

```
import kotlinx.coroutines.flow.toList
import org.springframework.stereotype.Service
@Service
class UserService(private val userRepository: UserRepository) {
    // 사용자 저장
    suspend fun saveUser(user: User): User {
        return userRepository.save(user)
    }

    // ID로 사용자 조회
    suspend fun getUserById(id: Long): User? {
        return userRepository.findById(id)
    }

    // 이름으로 사용자 조회
    suspend fun getUserByName(name: String): User? {
        return userRepository.findByName(name)
    }

    // 모든 사용자 조회
    suspend fun getAllUsers(): List<User> {
        return userRepository.findAll().toList() // Flow를 List로 변환
    }

    // 사용자 삭제
    suspend fun deleteUser(id: Long) {
        userRepository.deleteById(id)
    }
}
```

WebFlux와 R2DBC 기반 Database 연동

◆ 데이터베이스 연동 기능이 포함된 서비스의 REST Controller 예제

```
import org.springframework.web.bind.annotation.*
@RestController
@RequestMapping("/users")
class UserController(private val userService: UserService) {
    @PostMapping
    suspend fun createUser(@RequestBody user: User): User {
        return userService.saveUser(user)
    }
    @GetMapping("/{id}")
    suspend fun getUserById(@PathVariable id: Long): User? {
        return userService.getUserById(id)
    }
    @GetMapping
    suspend fun getAllUsers(): List<User> {
        return userService.getAllUsers()
    }
    @DeleteMapping("/{id}")
    suspend fun deleteUser(@PathVariable id: Long) {
        userService.deleteUser(id)
    }
}
```

Spring WebClient

◆ Spring WebClient

- Spring WebFlux와 함께 사용되는 Spring WebClient는 Spring 5에서 도입된 비동기식 및 반응형 HTTP 클라이언트
- Spring WebClient는 기존의 동기방식의 RestTemplate을 대체하며, 비동기 및 반응형 방식으로 설계되어 더 높은 성능과 유연성을 제공하며, 이는 특히 비동기 방식이 중요한 마이크로서비스 아키텍처나 고성능 시스템에서 유용함

WebFlux와 WebClient 기반 Multimedia Streaming 서비스 구현

Multimedia Streaming Service

◆ Multimedia Streaming Service 제공 시 고려사항

- 성능 최적화:
 - 파일 크기가 큰 경우 Flux를 사용하여 데이터를 chunk로 나누어 스트리밍해야 메모리 소비를 줄일 수 있음
 - ZeroCopyHttpOutputMessage를 사용하여 파일을 전송하는 효율성을 높일 수 있음
- Range 요청 처리:
 - 동영상 플레이어나 음악 플레이어는 Range 요청을 통해 특정 데이터 범위를 요청함
 - Range 요청 처리 기능을 지원하면 동영상 탐색 기능(스크러빙)을 구현할 수 있음
- 보안 및 인증:
 - 멀티미디어 파일이 민감한 정보인 경우 인증 및 권한 관리를 추가해야 함
- 클라이언트 처리:
 - 클라이언트가 수신한 데이터를 처리하고 재생하려면 적절한 뷰어 또는 플레이어와 통합해야 함

WebFlux와 WebClient 기반의 멀티미디어 스트리밍 서비스 구현

◆ WebFlux와 WebClient 기반의 멀티미디어 스트리밍 서비스 구현

- WebFlux와 WebClient를 사용하여 멀티미디어 스트리밍 서비스를 구현할 수 있음
- WebFlux의 non-blocking 특성 덕분에 서버가 높은 동시성을 유지하며 많은 클라이언트 요청을 처리할 수 있음

WebFlux, Kotlin Co-routine과 WebClient 기반의 멀티미디어 스트리밍 서비스 구현 프로젝트 구조

◆ Multimedia Streaming Service 프로젝트

The screenshot displays the IDE interface for the project 'ch13_063_WebFlux_WebClient_MultimediaStreaming'. The Project view on the left shows the following structure:

- ch13_063_WebFlux_WebClient_MultimediaStreaming
 - .gradle
 - .idea
 - build
 - gradle
 - src
 - main
 - kotlin
 - StreamingClient
 - StreamingServer
 - MediaHandler
 - MediaRouter
 - StreamingServer.kt
 - resources
 - application.properties
 - audio.mp3
 - video.mp4

The build.gradle file in the center shows the following configuration:

```

1 plugins {
2     kotlin("jvm") version "1.9.24"
3     kotlin("plugin.spring") version "1.9.24"
4     id("org.springframework.boot") version "3.2.5"
5     id("io.spring.dependency-management") version "1.1.5"
6 }
7
8 group = "WebFlux"
9 version = "1.0.0"
10
11 repositories {
12     mavenCentral()
13 }
14
15 dependencies {
16     // Web Server
17     implementation("org.springframework.boot:spring-boot-starter-webflux")
18
19     // Coroutines
20     implementation("org.jetbrains.kotlin:kotlinx-coroutines-core:1.7.3")
21     implementation("org.jetbrains.kotlin:kotlinx-coroutines-reactor:1.7.3")
22     implementation("io.projectreactor.kotlin:reactor-kotlin-extensions:1.2.2")
23 }
24
25 springBoot {
26     mainClass.set("WebFlux.StreamingServer.server.StreamingServerKt")
27 }
28
29 tasks.bootRun {
30     mainClass.set("WebFlux.StreamingServer.server.StreamingServerKt")
31 }
32

```

The Gradle view on the right shows the following tasks:

- ch13_063_WebFlux_WebClient_MultimediaStreaming
 - Tasks
 - application
 - bootRun
 - bootTestRun
 - build
 - build setup
 - documentation
 - help
 - other
 - verification
 - Dependencies
 - Run Configurations

build.gradle.kts, setting.gradle.kts

build.gradle.kts

```

plugins {
    kotlin("jvm") version "1.9.24"
    kotlin("plugin.spring") version "1.9.24"
    id("org.springframework.boot") version "3.2.5"
    id("io.spring.dependency-management") version "1.1.5"
}

group = "WebFlux"
version = "1.0.0"

repositories {
    mavenCentral()
}

dependencies {
    // Web Server
    implementation("org.springframework.boot:spring-boot-starter-webflux")

    // Coroutines
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor:1.7.3")
    implementation("io.projectreactor.kotlin:reactor-kotlin-extensions:1.2.2")
}

springBoot {
    mainClass.set("WebFlux.StreamingServer.server.StreamingServerKt")
}

tasks.bootRun {
    mainClass.set("WebFlux.StreamingServer.server.StreamingServerKt")
}
  
```

setting.gradle.kts

```

plugins {
    id("org.gradle.toolchains.foojay-resolver-convention") version "0.8.0"
}

rootProject.name = "ch13_063_WebFlux_WebClient_MultimediaStreaming"
  
```

StreamingServer.kt

```

/* StreamingServer.kt */
package WebFlux.StreamingServer.server

import org.slf4j.LoggerFactory
import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
class StreamingApplication

private val logger = LoggerFactory.getLogger(StreamingApplication::class.java)

fun main(args: Array<String>) {
    logger.info("Starting WebFlux Multimedia Streaming Server")

    try {
        val context = runApplication<StreamingApplication>(*args)
        val env = context.environment

        logger.info("WebFlux Multimedia Streaming Server started successfully")
        logger.info("Application name: {}", env.getProperty("spring.application.name", "MultimediaStreaming"))
        logger.info("Server port: {}", env.getProperty("server.port", "8080"))
        logger.info("Video URL: http://localhost:}/stream/video", env.getProperty("server.port", "8080"))
        logger.info("Audio URL: http://localhost:}/stream/audio", env.getProperty("server.port", "8080"))
    } catch (e: Exception) {
        logger.error("Failed to start WebFlux Multimedia Streaming Server", e)
        throw e
    }
}

```

MediaRouter.kt

```
/* MediaRouter.kt */  
package WebFlux.StreamingServer.server  
  
import org.springframework.context.annotation.Bean  
import org.springframework.context.annotation.Configuration  
import org.springframework.web.reactive.function.server.router  
  
@Configuration  
class MediaRouter {  
  
    @Bean  
    fun mediaRoutes(handler: MediaHandler) = router {  
        GET("/stream/video", handler::streamVideo)  
        HEAD("/stream/video", handler::headVideo)  
  
        GET("/stream/audio", handler::streamAudio)  
        HEAD("/stream/audio", handler::headAudio)  
    }  
}
```

MediaHandler.kt

```
/* MediaHandler.kt (1) */
package WebFlux.StreamingServer.server

import org.slf4j.LoggerFactory
import org.springframework.core.io.ClassPathResource
import org.springframework.core.io.Resource
import org.springframework.core.io.buffer.DataBuffer
import org.springframework.core.io.buffer.DataBufferUtils
import org.springframework.core.io.buffer.DefaultDataBufferFactory
import org.springframework.http.MediaType
import org.springframework.stereotype.Component
import org.springframework.web.reactive.function.server.ServerRequest
import org.springframework.web.reactive.function.server.ServerResponse
import reactor.core.publisher.Mono

@Component
class MediaHandler {

    private val logger = LoggerFactory.getLogger(MediaHandler::class.java)

    private val videoResource: Resource = ClassPathResource("video.mp4")
    private val audioResource: Resource = ClassPathResource("audio.mp3")

    fun streamVideo(request: ServerRequest): Mono<ServerResponse> {
        return streamResource(
            resource = videoResource,
            mediaType = MediaType.valueOf("video/mp4"),
            label = "video"
        )
    }

    fun streamAudio(request: ServerRequest): Mono<ServerResponse> {
        return streamResource(
            resource = audioResource,
            mediaType = MediaType.valueOf("audio/mpeg"),
            label = "audio"
        )
    }
}
```

```
/* MediaHandler.kt (2) */

fun headVideo(request: ServerRequest): Mono<ServerResponse> {
    return headResource(videoResource, MediaType.valueOf("video/mp4"), "video")
}

fun headAudio(request: ServerRequest): Mono<ServerResponse> {
    return headResource(audioResource, MediaType.valueOf("audio/mpeg"), "audio")
}

private fun streamResource(
    resource: Resource,
    mediaType: MediaType,
    label: String
): Mono<ServerResponse> {
    if (!resource.exists()) {
        logger.warn("{} resource was not found in classpath", label)
        return ServerResponse.notFound().build()
    }

    val contentLength = safeContentLength(resource)

    logger.info("Streaming {} resource: filename={}, size={} bytes",
        label, resource.filename, contentLength)

    val body = DataBufferUtils.read(
        resource, DefaultDataBufferFactory(), 64 * 1024
    )

    val responseBuilder = ServerResponse.ok()
        .contentType(mediaType)
        .headers { headers -> headers.set("Accept-Ranges", "bytes")
    }

    if (contentLength >= 0) {
        responseBuilder.contentLength(contentLength)
    }

    return responseBuilder.body(body, DataBuffer::class.java)
}
```

/ MediaHandler.kt (3) */*

```
private fun headResource(
    resource: Resource,
    mediaType: MediaType,
    label: String
): Mono<ServerResponse> {
    if (!resource.exists()) {
        logger.warn("{} resource was not found in classpath", label)
        return ServerResponse.notFound().build()
    }

    val contentLength = safeContentLength(resource)

    val responseBuilder = ServerResponse.ok()
        .contentType(mediaType)
        .headers { headers ->
            headers.set("Accept-Ranges", "bytes")
        }

    if (contentLength >= 0) {
        responseBuilder.contentLength(contentLength)
    }

    return responseBuilder.build()
}

private fun safeContentLength(resource: Resource): Long {
    return try {
        resource.contentLength()
    } catch (e: Exception) {
        logger.warn("Could not calculate content length for {}", resource.filename, e)
        -1
    }
}
```

StreamingClient.kt

```
/* StreamingClient.kt (1) */
package WebFlux.StreamingClient.StreamingClient

import kotlinx.coroutines.reactor.awaitSingleOrNull
import kotlinx.coroutines.runBlocking
import org.springframework.core.io.buffer.DataBuffer
import org.springframework.core.io.buffer.DataBufferUtils
import org.springframework.http.MediaType
import org.springframework.web.reactive.function.client.WebClient
import java.awt.Desktop
import java.io.File
import java.nio.file.StandardOpenOption
import java.util.concurrent.atomic.AtomicLong

fun main() = runBlocking {
    val baseUrl = "http://localhost:8080"
    val videoFile = File("downloaded_video.mp4")
    val audioFile = File("downloaded_audio.mp3")

    val client = WebClient.create(baseUrl)

    suspend fun download(uri: String, outputFile: File, mediaType: String) {
        println("▶ $uri start download ...")

        val totalSize = client.head()
            .uri(uri)
            .retrieve()
            .toBodilessEntity()
            .awaitSingleOrNull()
            ?.headers?.contentLength ?: -1

        val downloadedBytes = AtomicLong(0)
```

```
/* StreamingClient.kt (2) */

val responseFlux = client.get()
    .uri(uri)
    .accept(MediaType.valueOf(mediaType))
    .retrieve()
    .bodyToFlux(DataBuffer::class.java)
    .doOnNext { dataBuffer ->
        val size = dataBuffer.readableByteCount()
        val current = downloadedBytes.addAndGet(size.toLong())

        if (totalSize > 0) {
            val percent = current * 100 / totalSize
            print("\rdownloading ..... : $percent% ($current / $totalSize bytes)")
        } else {
            print("\rdownloaded bytes : $current bytes")
        }
    }

DataBufferUtils.write(
    responseFlux,
    outputFile.toPath(),
    StandardOpenOption.CREATE,
    StandardOpenOption.TRUNCATE_EXISTING,
    StandardOpenOption.WRITE
).awaitSingleOrNull()

println("\ndownload completed now: ${outputFile.absolutePath}")

if (Desktop.isDesktopSupported()) {
    println("▶ playing : ${outputFile.name}")
    Desktop.getDesktop().open(outputFile)

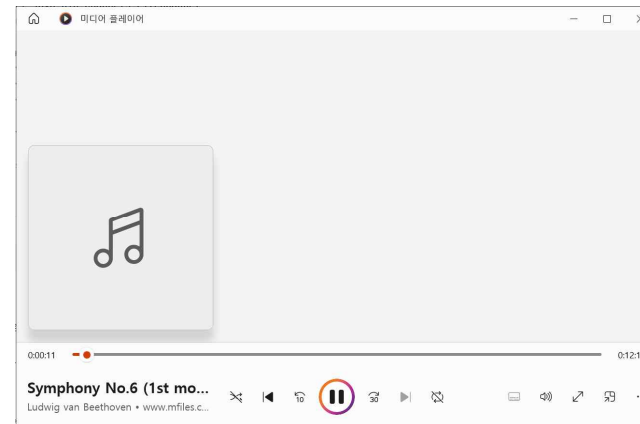
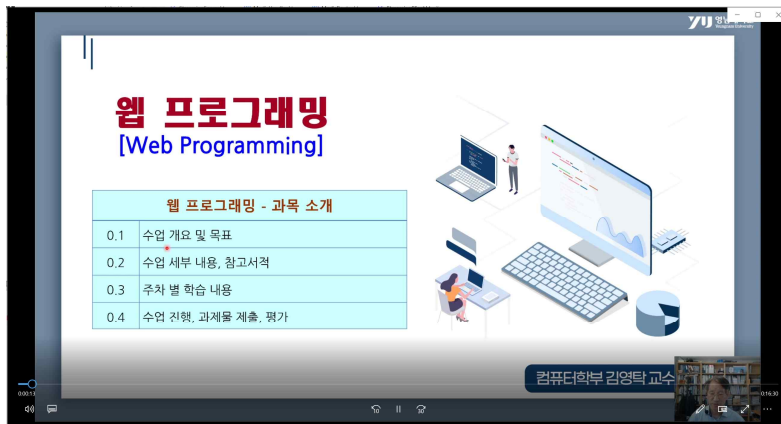
    println("⌚ press Enter key to continue next download after playing : ")
    readln()
}

download("/stream/video", videoFile, "video/mp4")
download("/stream/audio", audioFile, "audio/mpeg")
}
```

실행 결과

```
C:\Users\Onwer\.jdk\corretto-21.0.4\bin\java.exe ...
> /stream/video start download ...
downloaded bytes : 96579464 bytes
download completed now: F:\YTK-Lecture-2025\2025-1 웹프로그래밍\Kotlin프로그램\ch13-WebFlux, WebClient\ch13_063_WebFlux_WebClient_MultimediaStreaming\downloaded_video.mp4
> playing : downloaded_video.mp4
⏏ press Enter key to continue next download after playing :

> /stream/audio start download ...
downloaded bytes : 11650187 bytes
download completed now: F:\YTK-Lecture-2025\2025-1 웹프로그래밍\Kotlin프로그램\ch13-WebFlux, WebClient\ch13_063_WebFlux_WebClient_MultimediaStreaming\downloaded_audio.mp3
> playing : downloaded_audio.mp3
⏏ press Enter key to continue next download after playing :
```



Homework 13

Homework 13.1 – Spring Framework 기반 Bulletin Board

◆ Homework 13.1

- Spring Framework, SpringBoot와 MySQL DB를 사용하여 게시판 (Bulletin Board) 웹서비스를 구현하라.
- MVC (Model, View, Controller) 프로젝트 구조로 구현할 것
- 게시판을 사용하기 위한 사용자 등록 (register) 기능을 구현하고, 등록된 사용자는 MySQL DB에서 관리할 수 있도록 하라.
- 게시판을 사용하기 위한 사용자 login 기능을 구현하고, 성공적으로 login된 사용자가 게시판에 새로운 메시지를 등록할 수 있도록 하라.
- 새로운 message 등록을 위한 text 입력 필드와 post 버튼을 Web 페이지에 포함되도록 하라.
- 게시판에는 등록된 메시지의 제목, 저자 (author), 최종 update 일시가 표 형식으로 보여질 수 있도록 하라.
- 게시판에 나열된 메시지 중 본인이 작성한 message에 대한 편집(edit)과 삭제(delete) 기능이 제공될 수 있도록 하라.

The third screenshot shows the 'Bulletin Board (login as abc)' page. It features a table with the following data:

Seq	Message Title	Author	Last Update	Actions
1	Message of KKK 1 (updated)	KKK	2024-10-25T09:02:18.597813	
2	Hello message from author abc	abc	2024-10-25T09:04:58.650657	Edit Delete

Below the table, there is a form to post a new message:

Enter your message here

[Post](#)

Homework 13.2 – WebFlux, Kotlin Co-routine 및 WebClient 기반 Streaming 서비스 구현

◆ Homework 13.2

- WebFlux와 Kotlin Co-routing 기반으로 Audio 및 Video Streaming 서비스를 제공하는 Streaming Server를 구현하라.
- WebClient를 기반으로 Audio 및 Video Streaming 서비스를 제공하는 Streaming Client를 구현하라.
- 기능 시험을 위하여 video.mp4와 audio.mp3를 함께 준비할 것.

◆ Homework 13.2 Project 구조

The screenshot displays an IDE interface with the following components:

- Project View (Left):** Shows the project structure. A red dashed box highlights the `src/main/kotlin` directory, which contains the following files:
 - `StreamingClient.kt`
 - `StreamingServer.kt`
 - `MediaHandler.kt`
 - `MediaRouter.kt`
 - `StreamingServer.kt`
- Code Editor (Center):** Displays the `build.gradle.kts` file for the project `ch13_063_WebFlux_WebClient_MultimediaStreaming`. The code is as follows:


```
1 plugins {
2     kotlin("jvm") version "1.9.24"
3     kotlin("plugin.spring") version "1.9.24"
4     id("org.springframework.boot") version "3.2.5"
5     id("io.spring.dependency-management") version "1.1.5"
6 }
7
8 group = "WebFlux"
9 version = "1.0.0"
10
11 repositories {
12     mavenCentral()
13 }
14
15 dependencies {
16     // Web Server
17     implementation("org.springframework.boot:spring-boot-starter-webflux")
18
19     // Coroutines
20     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
21     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor:1.7.3")
22     implementation("io.projectreactor.kotlin:reactor-kotlin-extensions:1.2.2")
23 }
24
25 springBoot {
26     mainClass.set("WebFlux.StreamingServer.server.StreamingServerKt")
27 }
28
29 tasks.bootRun {
30     mainClass.set("WebFlux.StreamingServer.server.StreamingServerKt")
31 }
32 }
```
- Gradle View (Right):** Shows the task list for the project, including `bootRun`, `bootTestRun`, `build`, `build setup`, `documentation`, `help`, `other`, `verification`, `Dependencies`, and `Run Configurations`.

◆ Homework 13.2 실행결과 예시

```
C:\Users\Onwer\.jdk\corretto-21.0.4\bin\java.exe ...
▶ /stream/video start download ...
downloaded bytes : 96579464 bytes
download completed now: F:\YTK-Lecture-2025\2025-1 웹프로그래밍\Kotlin프로그래밍\ch13-WebFlux, WebClient\ch13_063_WebFlux_WebClient_MultimediaStreaming\downloaded_video.mp4
▶ playing : downloaded_video.mp4
⌘ press Enter key to continue next download after playing :

▶ /stream/audio start download ...
downloaded bytes : 11650187 bytes
download completed now: F:\YTK-Lecture-2025\2025-1 웹프로그래밍\Kotlin프로그래밍\ch13-WebFlux, WebClient\ch13_063_WebFlux_WebClient_MultimediaStreaming\downloaded_audio.mp3
▶ playing : downloaded_audio.mp3
⌘ press Enter key to continue next download after playing :
```

